



Global Knowledge®
a skillsoft company

Automatisation de la configuration avec Ansible



ANSIBLE

Agenda

1. **La gestion de la Configuration**
2. **Positionnement de Ansible**
3. **Installation et configuration**
4. **Les commandes Ad-Hoc**
5. **Les playbooks**
6. **Les rôles et les includes**
7. **Ansible Galaxy**
8. **Sécurité**
9. **Ansible Tower**



Global Knowledge®
a skillsoft company

La Gestion de Configuration



Global Knowledge®
a skillsoft company

Gestion de Configuration

Selon Wikipedia : La gestion des configurations est définie comme suit :

- « La gestion de configuration consiste à gérer la description technique d'un système (et de ses divers composants), ainsi qu'à gérer l'ensemble des modifications apportées au cours de l'évolution du système. »

En d'autres termes, il s'agit de l'ensemble des processus permettant d'assurer la conformité de la configuration d'un système aux exigences, tout au long de son cycle de vie.

La gestion de configuration est utilisée pour la configuration de systèmes complexes



Gestion de Configuration

Résout les problèmes d'administration systèmes de grands parcs :

- Tâches répétitives
- Supervision de l'état du parc
- Audit
- etc.



Gestion de Configuration

Méthodes possibles d'administration de grands parcs

- Manuellement
- Scripts sh
- Outils de Gestion de Configuration



Gestion de Configuration

Méthodes possibles d'administration de grands parcs

- **Manuellement**
 - Longue (surtout avec plusieurs machines)
 - Répétitive
 - Risque d'erreurs (l'erreur est humaine)



Gestion de Configuration

Méthodes possibles d'administration de grands parcs

- **Boucle SSH (script)**
 - Long!
 - Comment vérifier les résultats ?
 - Reprise sur erreur
 - Prérequis: s'assurer que tous les systèmes sont iso (ou s'assurer que tous les cas de figure sont gérés)
- **Update.sh (script au boot ou par cron/scheduler)**
 - Comment s'assurer que tous les nœuds ont bien exécuté le script?
 - Mêmes problèmes
 - Comment distinguer les nœuds?



Gestion de Configuration

Méthodes possibles d'administration de grands parcs

- **CM Tools**
 - Ansible
 - Puppet
 - ...
- Avantages:
 - Déclaratives
 - Idempotentes
 - Exécution parallèle
 - Réduire le temps de préparation



Systemes de Gestion de Configuration

Configuration Management Tools

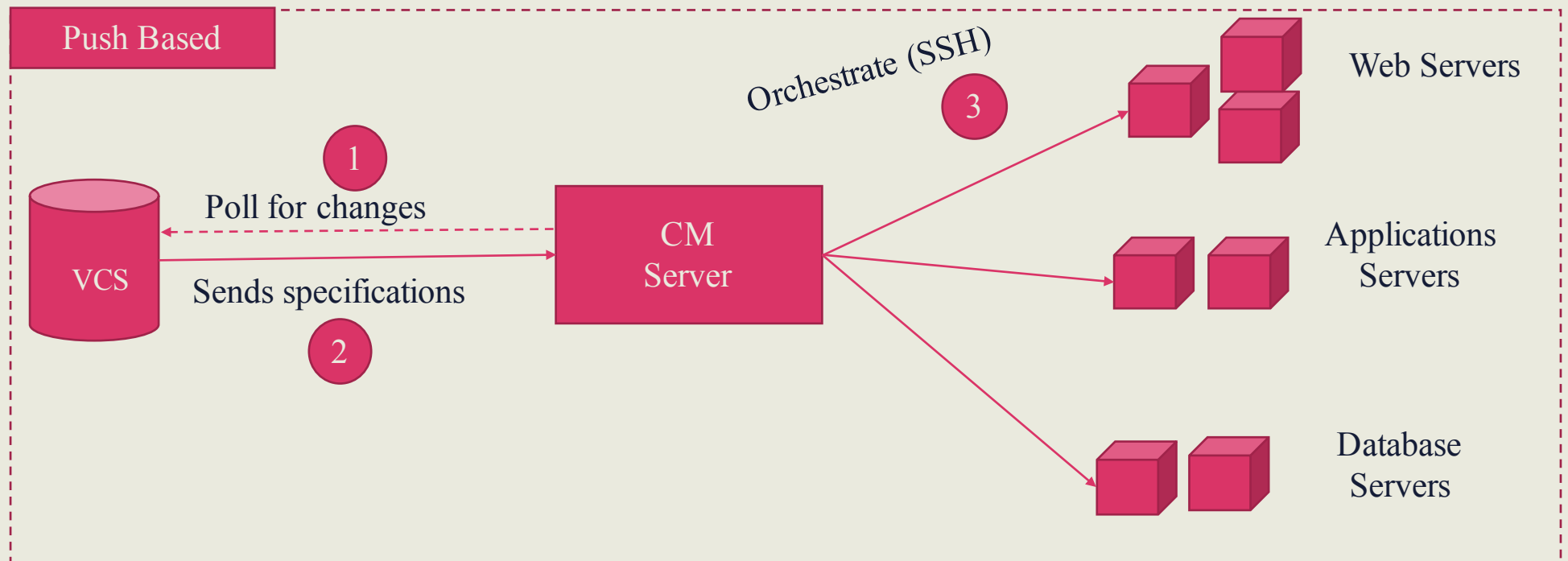
Deux mécanismes

- Push based
- Pull based



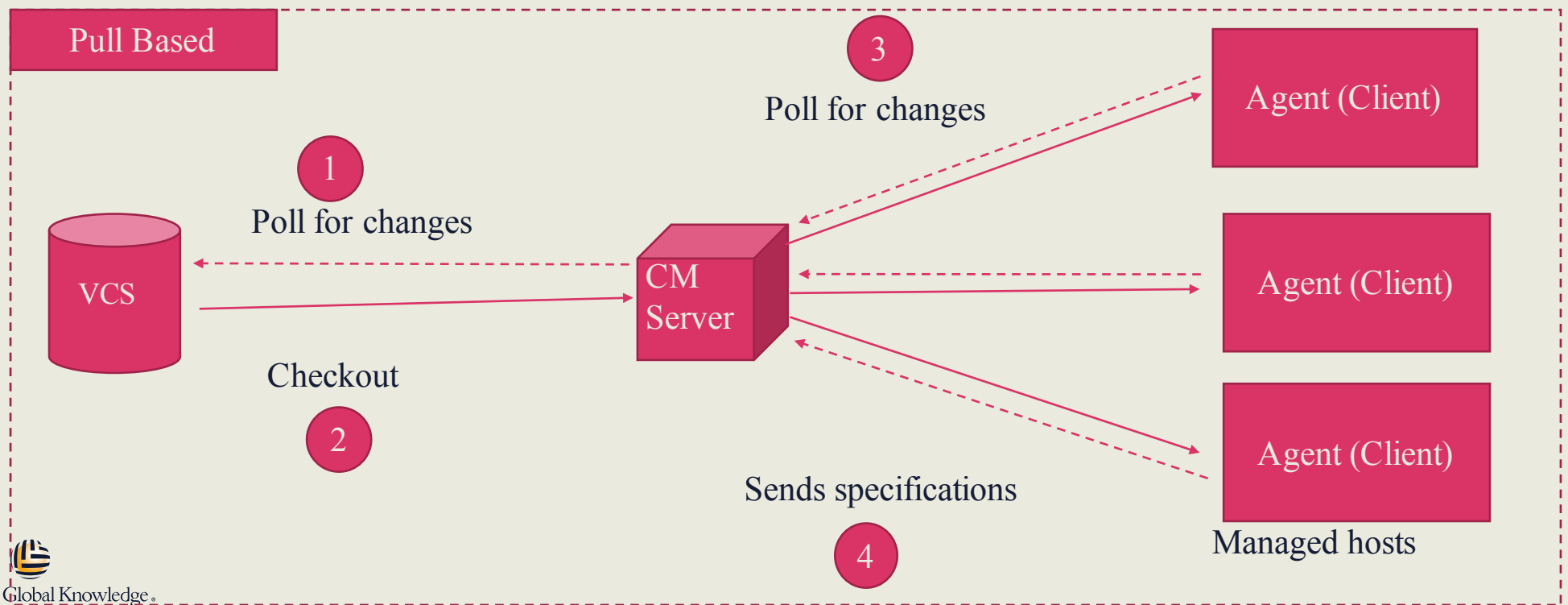
Systemes de Gestion de Configuration

Push Based vs Pull Based



Systemes de Gestion de Configuration

Push Based vs Pull Based



Systèmes de Gestion de Configuration

Modèle Push vs Modèle Pull

Des outils comme Puppet et Chef sont basés sur le modèle Pull:

- Les agents sur le serveur vérifient périodiquement les informations de configuration du serveur central (maître).

Ansible est basé sur le modèle Push:

- Le serveur central envoie les informations de configuration sur les serveurs cibles
- Vous contrôlez le moment où les modifications sont effectuées sur les serveurs



Ansible



Global Knowledge®
a skillsoft company



Qu'est ce que Ansible?

Ansible est un moteur d'automatisation de

- Provisionnement
- Configuration
- Déploiement d'applications
- Orchestration interserveurs
- ...



Qu'est ce que Ansible?

Gestion de configuration

- Configuration logicielle
- Historique des changement de configuration
- Reproduction de la même configuration sur plusieurs machines



Qu'est ce que Ansible?

Provisioning

- tous les paquetages nécessaires et tous les logiciels sont téléchargés et installés sur votre ordinateur afin d'exécuter votre application



Qu'est ce que Ansible?

utilise des '**Playbooks**' pour

- déployer,
- gérer,
- créer,
- tester
- configurer



Global Knowledge®
a skillsoft company

Pourquoi encore un autre outil?

“I wrote Ansible because none of the existing tools fit my brain. I wanted a tool that I could not use for 6 months, come back later, and still remember how it worked.”

Michael DeHaan, Ansible Founder

“We need to do a rolling deployment of changes that have certain dependencies (including external services). With Ansible this becomes trivial.

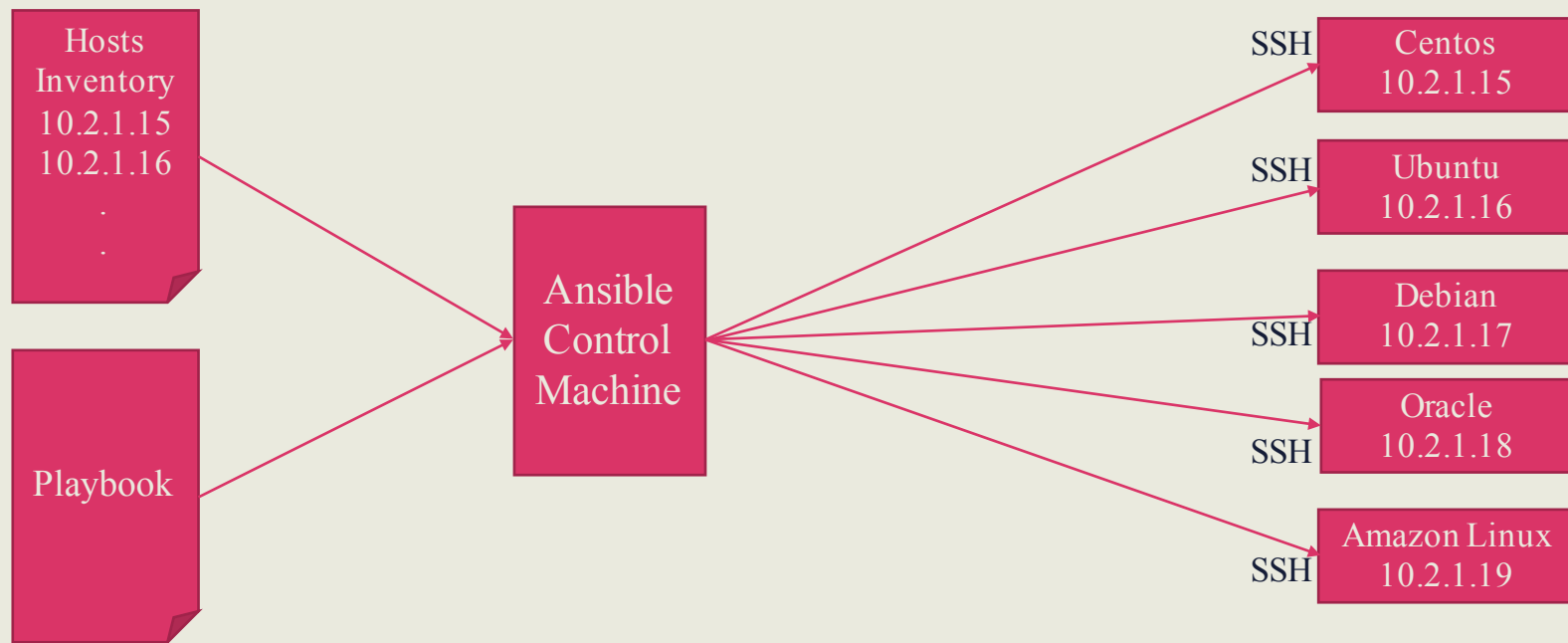
Puppet on the other hand feels like the Wild West.”

User IUseRhetoric on [reddit.com](https://www.reddit.com)



Global Knowledge®
a skillsoft company

Ansible : Architecture Agentless



Vocabulaire d'Ansible

- **Machine de Contrôle:** la machine sur laquelle Ansible est installée.
- **Inventaire :** les serveurs gérés.
- **Tâches (tasks) :** une procédure à exécuter
- **Module :** le script qui s'exécute sur le système géré.
- **Playbooks :** les serveurs cibles et les tâches à exécuter.
- **Role :** partage et réutilisation des configurations.
- **Facts :** Des informations à propos du système géré
- **Handlers:** Exécution conditionnée



Caractéristiques Ansible

- Sans agent: pas besoin d'installation ni de gestion d'agent
- Construit avec Python et fournit donc beaucoup de fonctionnalités de Python
- Utilise SSH pour des connexions sécurisées
- Suit le modèle Push pour l'envoi de configurations
- Très facile et rapide à installer, exigences minimales

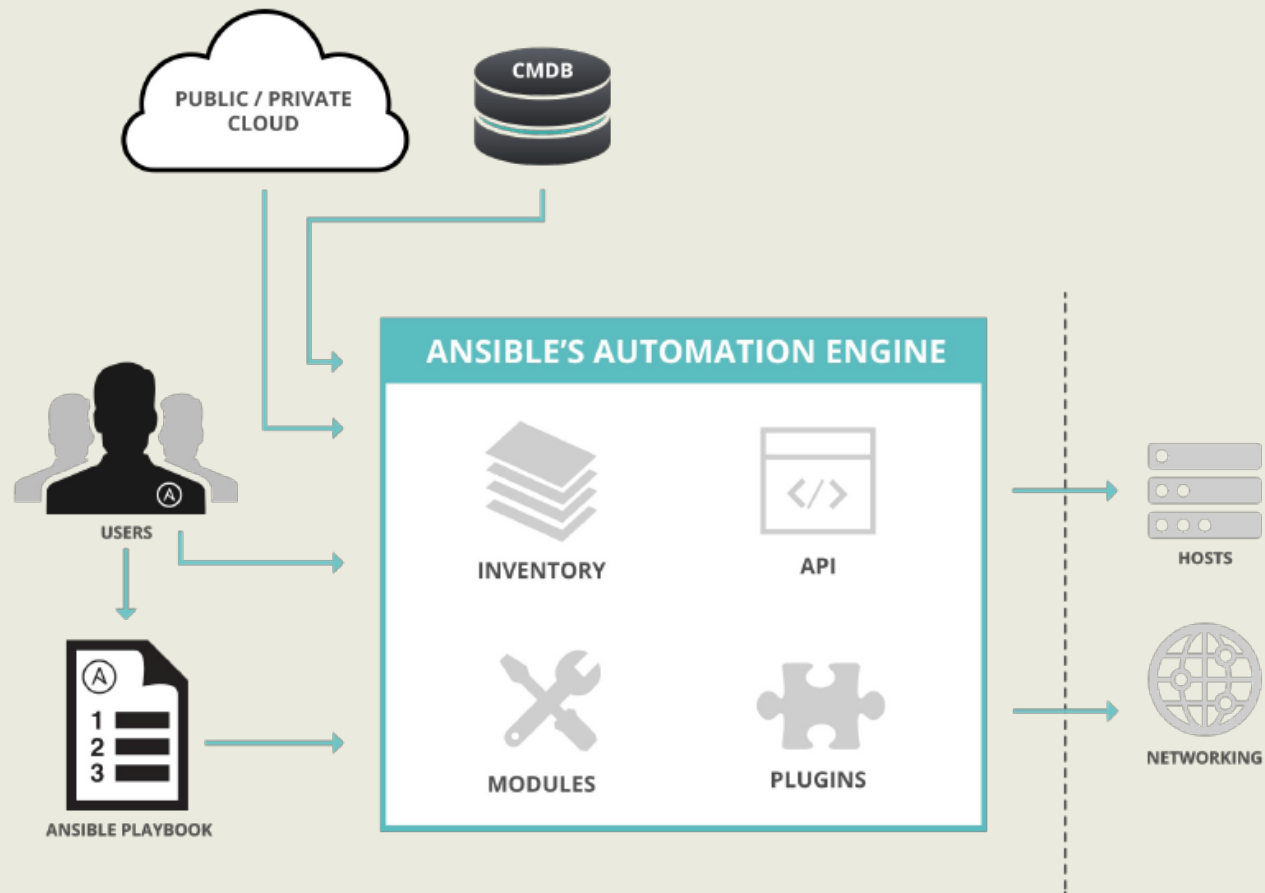


Caractéristiques Ansible

- Utilise YAML pour la définition des configurations
- Fichiers de configuration simples à écrire et à lire : pas de programmation complexe!
- Notion de Playbook : ensemble de commandes à appliquer pour une configuration souhaitée
- Notion de Role : niveau d'abstraction au dessus d'un playbook, générique et réutilisable



Ansible Architecture

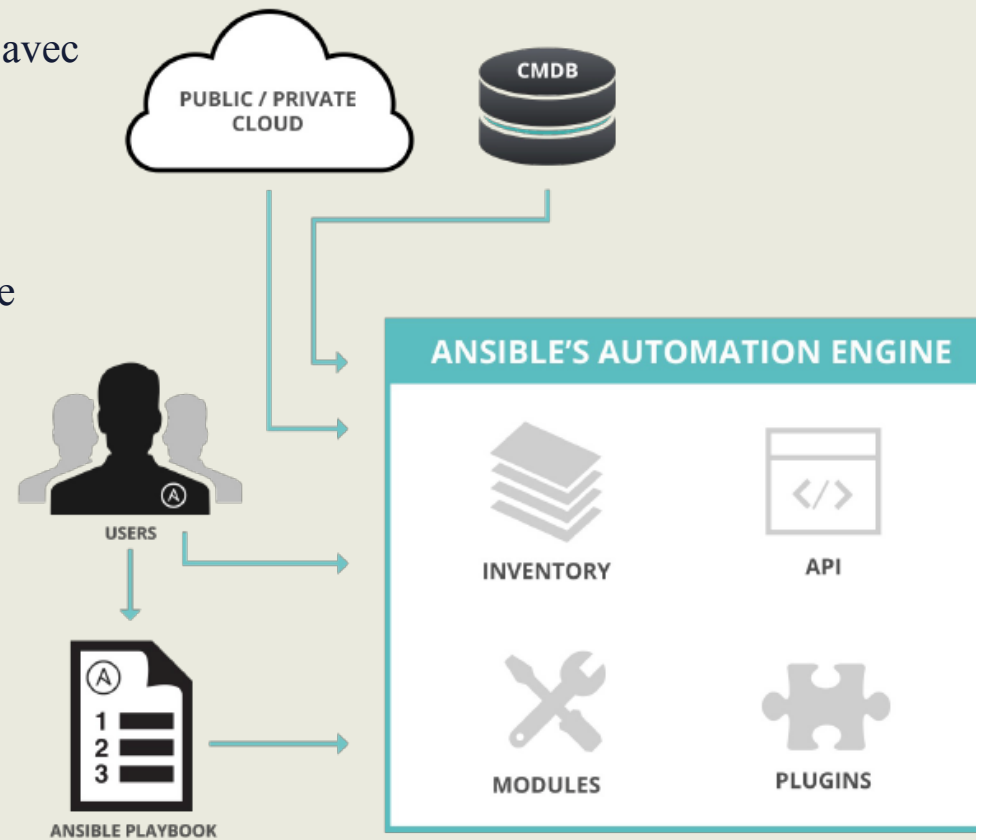


Architecture d'Ansible

Le moteur d'automatisation Ansible interagit directement avec les utilisateurs:

- écrire des playbooks à exécuter le moteur d'automatisation Ansible .

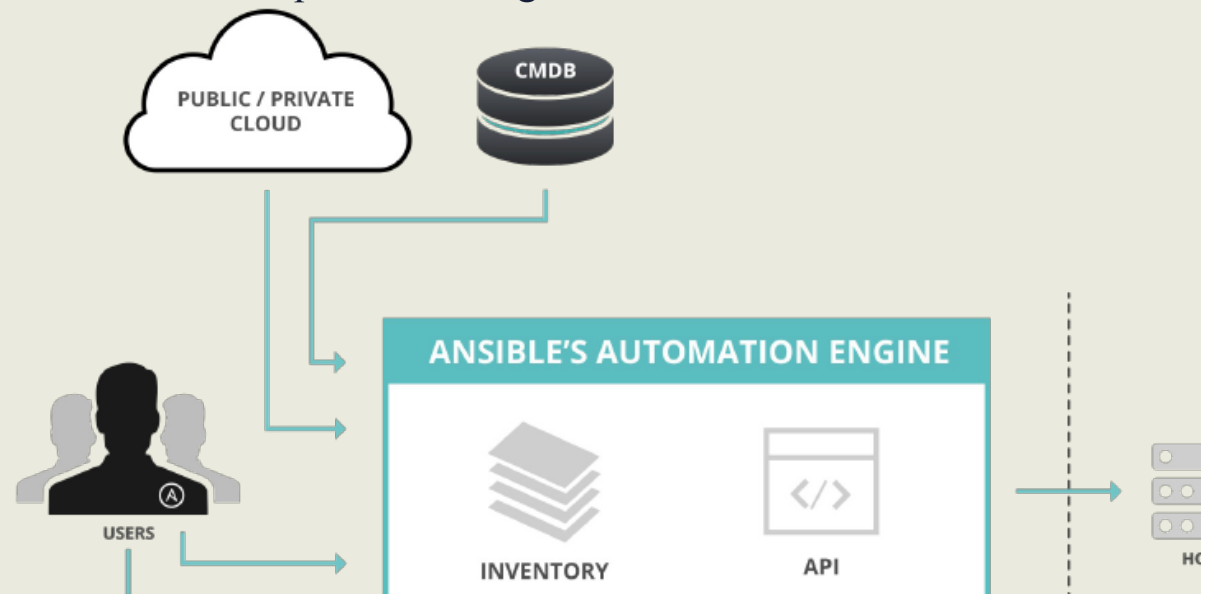
Il interagit également avec les services Cloud et la base de données de gestion de la configuration (CMDB).



Architecture d'Ansible

CMDB

- La CMDB fournit un lieu de stockage pour les informations relatives aux actifs informatiques et autres éléments de configuration.
- Les outils de découverte et d'importation des données sont généralement utilisés pour identifier les éléments de configuration dans l'environnement informatique et les intégrer dans la CMDB.

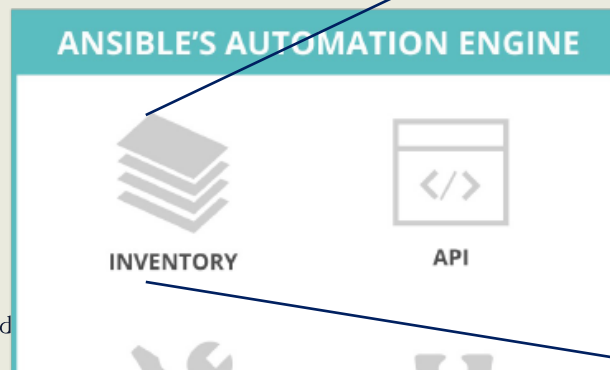


Architecture d'Ansible

Hosts Inventory

L'inventaire est une collection d'hôtes (nœuds) avec des données et des regroupements que Ansible peut gérer.

L'inventaire par défaut est `/etc/ansible/hosts`



Architecture d'Ansible

Exemple: Static Inventory

Group

```
# prod inventory
```

```
[balancers]
```

```
www.example.com
```

Host

```
[webservers]
```

```
www[0-9].example.com
```

Numeric Range

```
[dbservers]
```

```
db[a:f].example.com
```

Alphabetic Range

```
[monitoring]
```

```
dynatrace.example.com
```

ANSIBLE'S AUTOMATION ENGINE



INVENTORY



API



MODULES



PLUGINS

Global Knowledge.
a skillsoft company

Architecture d'Ansible

Dynamic Inventories

Utilisation de plusieurs fichiers d'inventaire simultanément et extraction des inventaires à partir de sources dynamiques ou de cloud:

- Cloud: Amazon, DigitalOcean, Google, OpenShift, OpenStack, etc.
- Distributed Information Services: LDAP, etcd, etc.

<https://github.com/ansible-collections/community.general/tree/main/plugins/inventory>



Architecture d'Ansible

Les API

Les API REST : fournir un accès aux ressources via des chemins URI

Ansible Tower : Rendre Ansible utilisable via un service Web.



Global Knowledge®
a skillsoft company

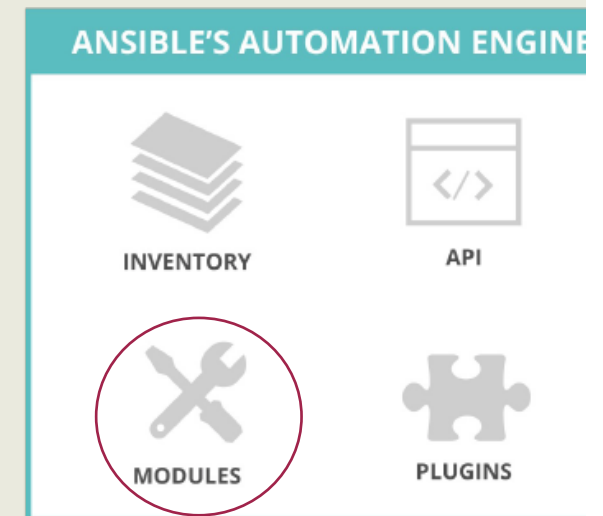
Architecture d'Ansible

Ansible Modules

- Les modules (également appelés "task plugins" ou "library plugins") sont ceux qui sont réellement exécutés dans un playbook
- Ce sont des scripts fournis avec Ansible et effectuent des actions sur les hôtes
- Exemples:
 - **apt**: install and remove packages using the apt package manager
 - **copy**: copies a file from local machine to hosts
 - **file**: sets the attribute of a file, symlink, or directory
 - **service**: starts, stops or restarts a service



Global Knowledge®
a skillsoft company



Architecture d'Ansible

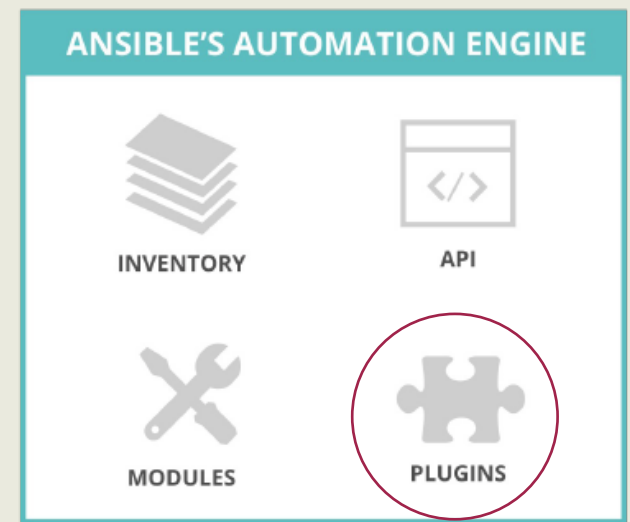
Les plugins

plugins d'Action :

- agissent conjointement avec les modules pour exécuter les actions requises par les tâches du playbook
- Exécution automatiquement en arrière-plan

plugins de cache:

- implémentation du mécanisme de cache d'arrière-plan
- Permettant à Ansible de stocker des facts collectés
- stocker des données source d'inventaire sans avoir récupérer les données à partir de la source.



Architecture d'Ansible

Les plugins

Callback plugins:

- Permettent de connecter à des événements Ansible afin d'affichage le journal.

Plugins de rappel:

- permettant d'ajouter de nouveaux comportements à Ansible lors de la réponse à des événements

Plugins de connexion:

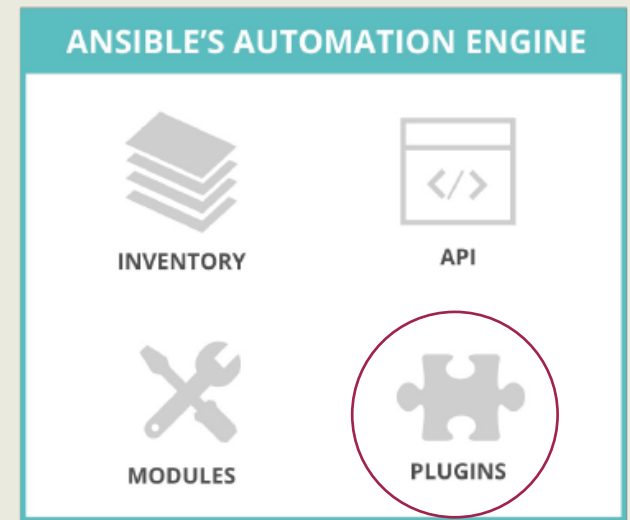
- permettant de se connecter aux hôtes cibles pour lui permettre d'exécuter des tâches.

Les plugins de recherche:

- permettant d'accéder à des données provenant de sources extérieures.



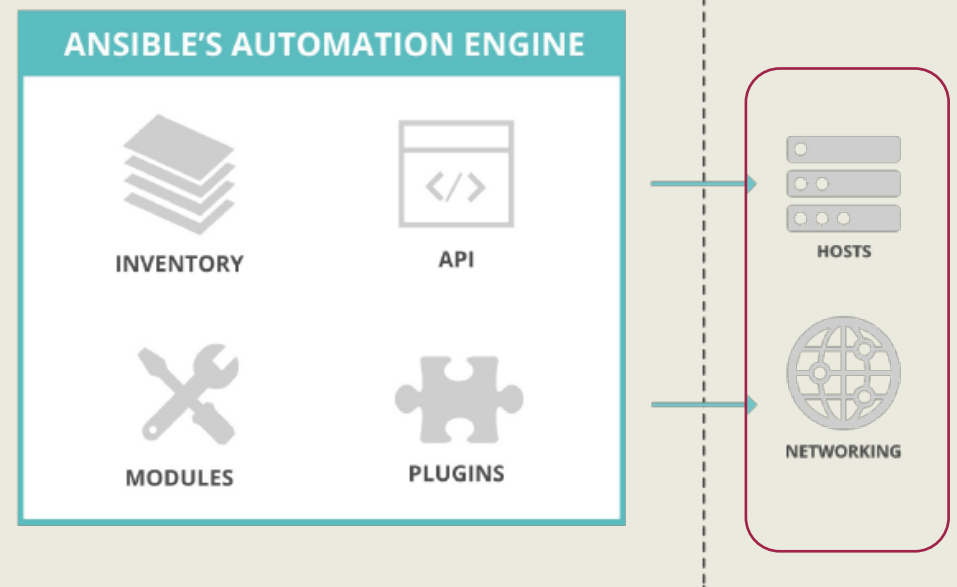
Global Knowledge®
a skillsoft company



Architecture d'Ansible

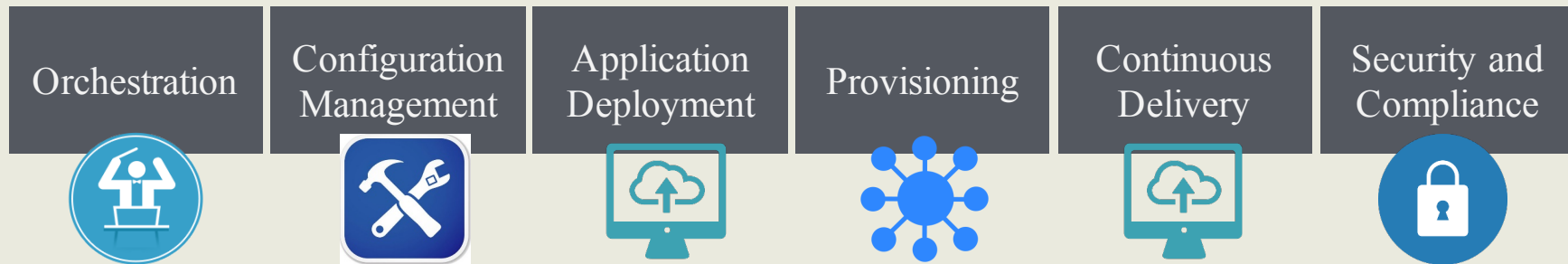
Les cibles Ansibles

- **Les hôtes système:** N'importe quelle machine physique ou virtuelle, avec un OS Linux / Windows ...
- **Les hôtes réseau:** Les équipements réseaux ayant un accès de configuration à distance (Routeurs, Switchers, Points d'accès, firewalls physiques ...)

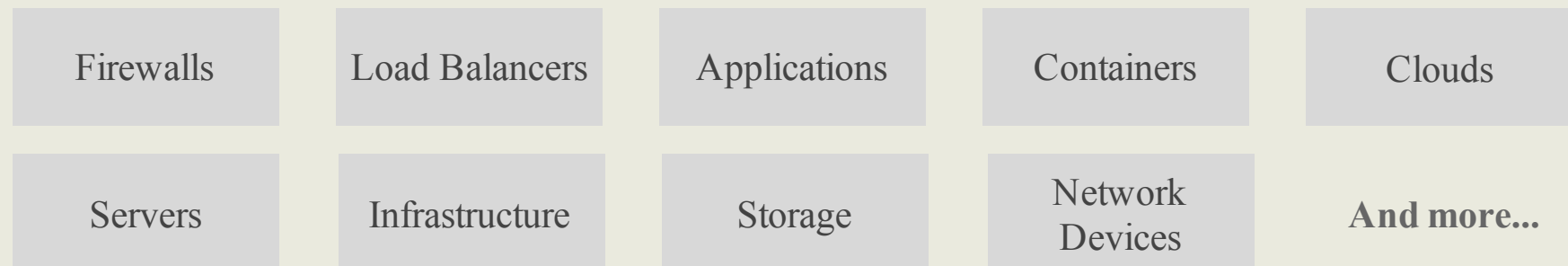


Les Fonctionnalités d'Ansible

Do this...



On these...

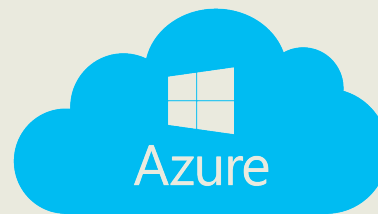


Global Knowledge®
a skillsoft company

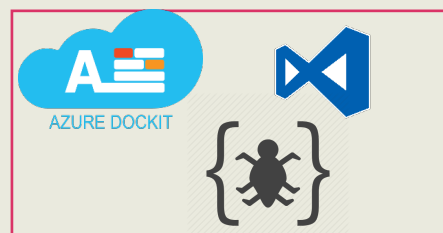
Les Fonctionnalités d'Ansible

Provisioning

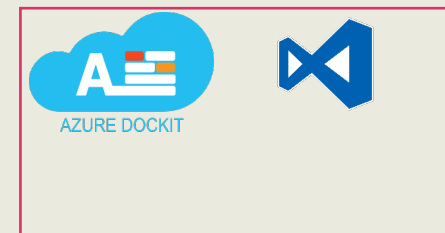
Provisioning a Python web application hosted on Cloud



Coding



Testing



Deploying



Global Knowledge®
a skillsoft company

Les Fonctionnalités d'Ansible

Provisioning

- Rationaliser le processus d'amorçage et de démarrage des serveurs bare-metal ou des machines virtuelles, ou création des instances virtuelles à partir de modèles.
- Provisionnement des dernières plates-formes de cloud, hôtes et hyperviseurs virtualisés, périphériques réseau et serveurs sans système d'exploitation.
- devient l'outil de connexion de tous vos pipelines de processus, en intégrant automatiquement l'infrastructure à la gestion quotidienne



Les Fonctionnalités d'Ansible

Configuration Management

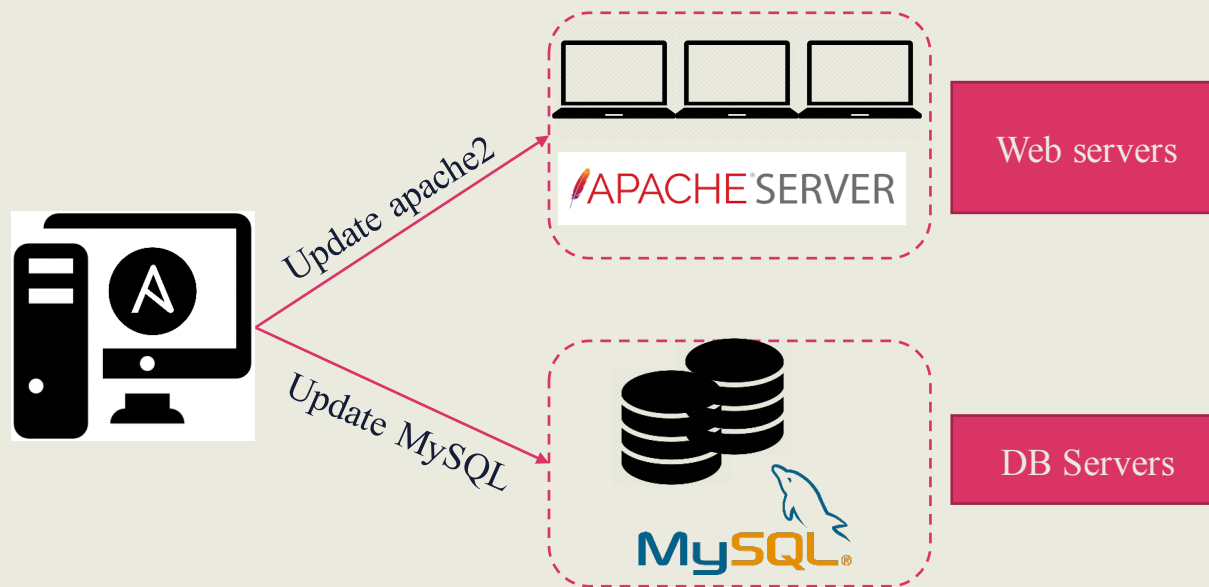
- Établir et maintenir la cohérence de configuration en enregistrant et en mettant à jour des informations détaillées décrivant le matériel et les logiciels d'une entreprise.
- Ces informations incluent les versions et les mises à jour appliquées aux packages de logiciels installés.
- Les emplacements et adresses réseaux des périphériques matériels.



Les Fonctionnalités d'Ansible

Configuration Management

LAMP Stack



Global Knowledge®
a skillsoft company

Les Fonctionnalités d'Ansible

Déploiement d'applications

Définir l'application avec Ansible

Génération du déploiement avec Ansible

Gérer efficacement le cycle de vie complet de l'application --> du développement à la production.



Global Knowledge®
a skillsoft company

Les Fonctionnalités d'Ansible

Déploiement d'applications:

Par exemple

Pour déployer le moteur de servlet par défaut. Un certain nombre d'étapes doivent être franchies pour déployer le moteur.

- Déplacer une application .war d'un répertoire dropins vers un répertoire d'applications
- Ajouter un fichier server.xml
- Accédez à la page Web pour voir votre application



Global Knowledge®
a skillsoft company

Les Fonctionnalités d'Ansible

Sécurité et conformité:

Automatiser le processus d'analyse et de correction de la stratégie de sécurité

- Configuration d'informations de sécurité une fois dans la machine de contrôle
- Déploiement et intégration automatiquement dans tous les autres nœuds.
- Cryptage des informations d'identification récupérées (identifiant de l'utilisateur admin et mots de passe)



Les Fonctionnalités d'Ansible

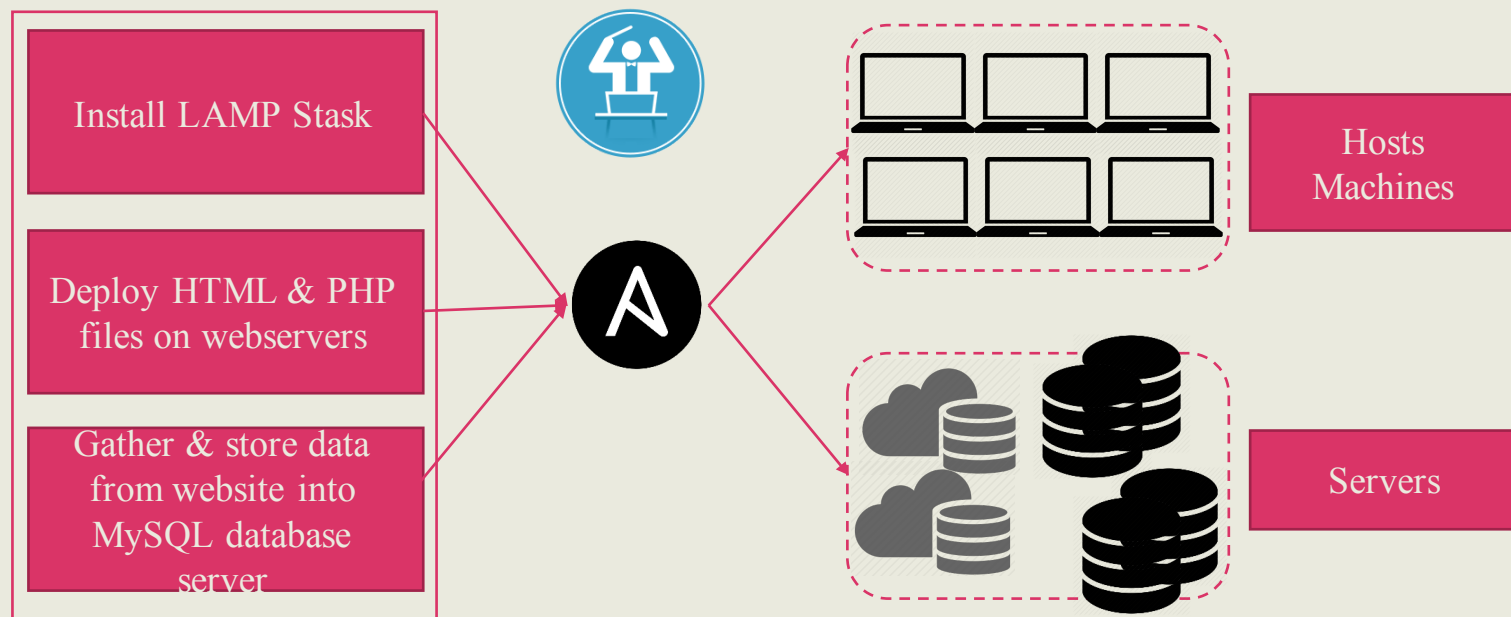
Orchestration

- Décrire comment automatiser un processus ou un workflow constitué de nombreuses étapes réalisées sur plusieurs systèmes disparates.
- Définir l'interaction de plusieurs configurations
- Assure que les éléments disparates peuvent être gérés dans leur ensemble.
- Création d'infrastructure alignée sur les applications qui peut être augmentée ou réduite en fonction des besoins de chaque application.



Les Fonctionnalités d'Ansible

Orchestration



Ansible Hands-on



Global Knowledge®
a skillsoft company

Installation

- Ansible est un outil disponible en ligne de commande.
- L'automatisation de configuration de système et le déploiement de logiciels est son objectif principal.
- Installable sur toute distribution Linux
 - Source code disponible sur github
 - Package RPM / DPKG téléchargeable
 - Référentiel Yum / Apt
 - Référentiel Pip
- Pour tout type d'installation de Ansible, consulter la documentation sur https://docs.ansible.com/ansible/latest/installation_guide/intro_installation.html



Installation

RHEL et CentOS

- Acquérir Ansible sur RHEL et CentOS en installant à partir du canal Ansible
 - Il faut activer le Ansible Engine repository:
\$ `subscription-manager repos --enable=rhel-7-server-ansible-2.x-rpms`
- OU installer le EPEL repository
\$ `yum install https://dl.fedoraproject.org/pub/epel/epel-release-latest-7.noarch.rpm`
- Puis, installer par yum:
\$ `sudo yum install ansible`



Installation

Debian / Ubuntu

- Ansible sur Ubuntu
 - Acquérir Ansible sur Ubuntu en utilisant le PPA Ansible

```
$ sudo add-apt-repository ppa:ansible/ansible  
$ sudo apt update  
$ sudo apt install ansible
```



Installation

pip

- Installer Ansible avec “pip”

- Installer les paquets Python

```
$ curl https://bootstrap.pypa.io/pip/2.7/get-pip.py -o get-pip.py
```

```
$ python get-pip.py --user
```

- Installer Ansible avec pip

```
$ python -m pip install --user ansible
```



Installation

Tester l'installation Ansible en récupérant sa version à l'aide de la commande suivante.

```
$ ansible --version
```



Global Knowledge®
a skillsoft company

Accès SSH

Configuration de l'accès SSH aux hôtes Ansible

La machine de contrôle Ansible contrôle les nœuds gérés via SSH.

L'authentification SSH peut être basée sur:

- Un mot de passe,
- Une paire de clés SSH.

Sous Linux, la configuration de l'accès SSH est dans le fichier:

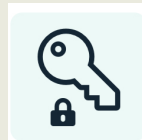
- `/etc/ssh/sshd_config`



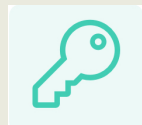
Accès SSH: configuration

Générer une pair de clés SSH

- `ssh-keygen`



Private key



Public key

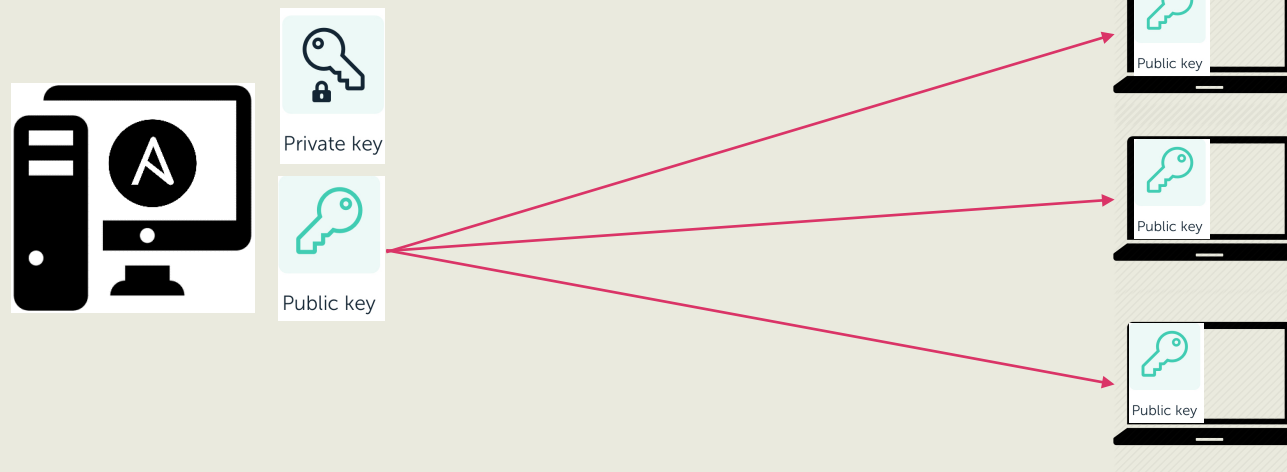


Global Knowledge®
a skillsoft company

Accès SSH: configuration

Installer la clé SSH dans les nœuds gérés

- Par copie manuelle de la clé publique dans le fichier « `~/.ssh/authorized_keys` »
- Par copie automatique en utilisant la commande « **ssh-copy-id** »



Accès SSH: configuration

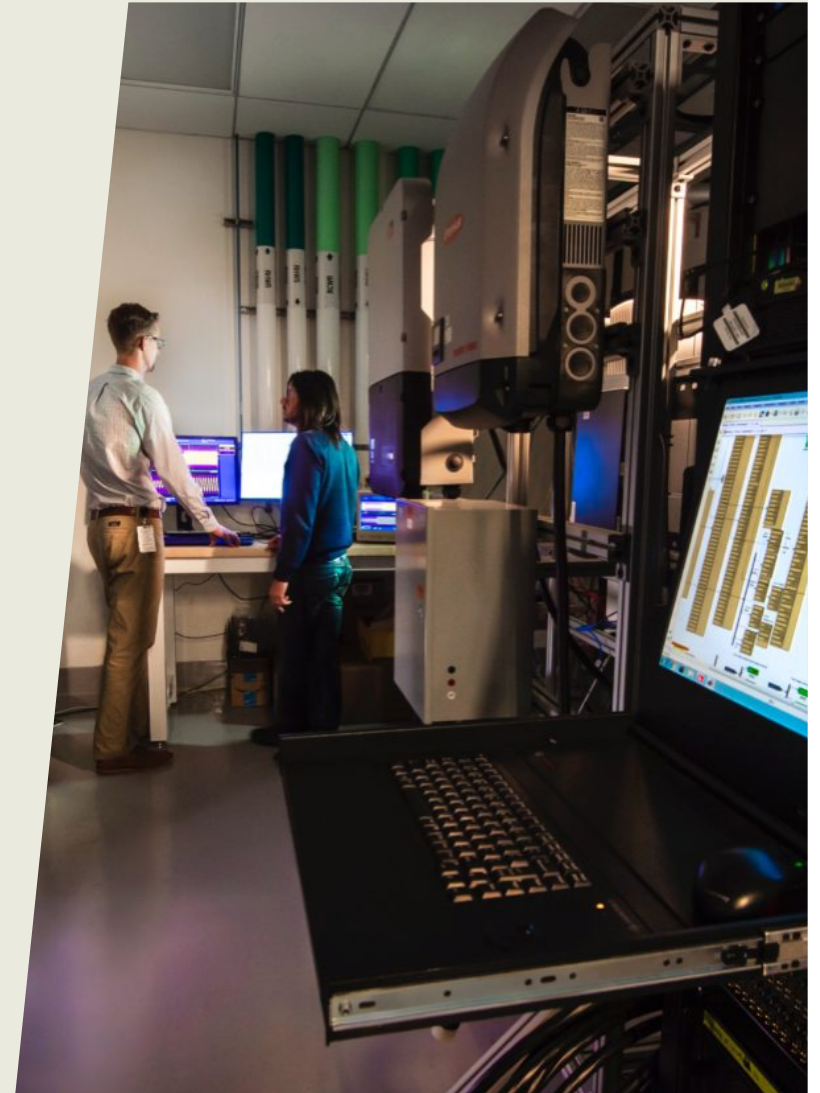
- Sur le serveur Ansible, afficher la clé publique SSH de votre utilisateur:
`$ cat ~/.ssh/id_rsa.pub`
- Sur le nœud géré, ouvrez le fichier `authorized_keys`:
`$ vi ~/.ssh/authorized_keys`
- collez votre clé SSH de l'utilisateur du serveur Ansible, puis enregistrez le fichier.
- Testez la connexion ssh sans mot de passe à partir du serveur:
`$ ssh user@node_ip`



Lab# 1: Installation de Ansible



Global Knowledge®
a skillsoft company



Configurer Ansible



Global Knowledge®
a skillsoft company

Configurer Ansible

Ansible prend en charge plusieurs sources pour configurer son comportement:

- un fichier ini nommé ansible.cfg,
- des variables d'environnement,
- des options de ligne de commande,
- des mots-clés dans le playbook
- des variables



Configurer Ansible:

Le fichier de configuration

Des changements dans la configuration peuvent être effectués dans:

- Variable d'environnement: `ANSIBLE_CONFIG`
- `ansible.cfg` (dans le répertoire courant)
- `~/.ansible.cfg` (dans le home directory)
- `/etc/ansible/ansible.cfg`

Une liste exhaustive des options de configuration:

- https://docs.ansible.com/ansible/latest/reference_appendices/config.html#common-options



Configurer Ansible

le fichier de configuration

Le fichiers de configuration est divisé en sections repérées par un mot entre crochets:

- [defaults]
- [ssh_connection]
- [selinux]
- [inventory]
- ...



Configurer Ansible

Le fichier de configuration

Exemples de configuration:

```
[defaults]
inventory = /etc/ansible/hosts
forks = 5
sudo_user = root
ask_sudo_pass = True
ask_pass = True
remote_port = 2200
```



Configurer Ansible

Le fichier de configuration

Exemples de configuration:

```
[defaults]
# smart - gather by default, but don't regather if already gathered
# implicit - gather by default, turn off with gather_facts: False
# explicit - do not gather by default, must say gather_facts: True
gathering = implicit

# uncomment this to disable SSH key host checking
host_key_checking = False
```



Configurer Ansible

Le fichier de configuration

Exemples de configuration:

```
# default user to use for playbooks if user is not specified
# (/usr/bin/ansible will use current user as default)
remote_user = vagrant

# logging is off by default unless this path is defined
# if so defined, consider logrotate
log_path = /var/log/ansible.log
```



Configurer Ansible

Le fichier de configuration

La commande **ansible-config** permet de consulter la configuration de Ansible

Les sous commande suivantes sont disponibles:

- **list** : Print all config options
- **dump** : Dump configuration
- **view** : View configuration file



Ansible Inventory

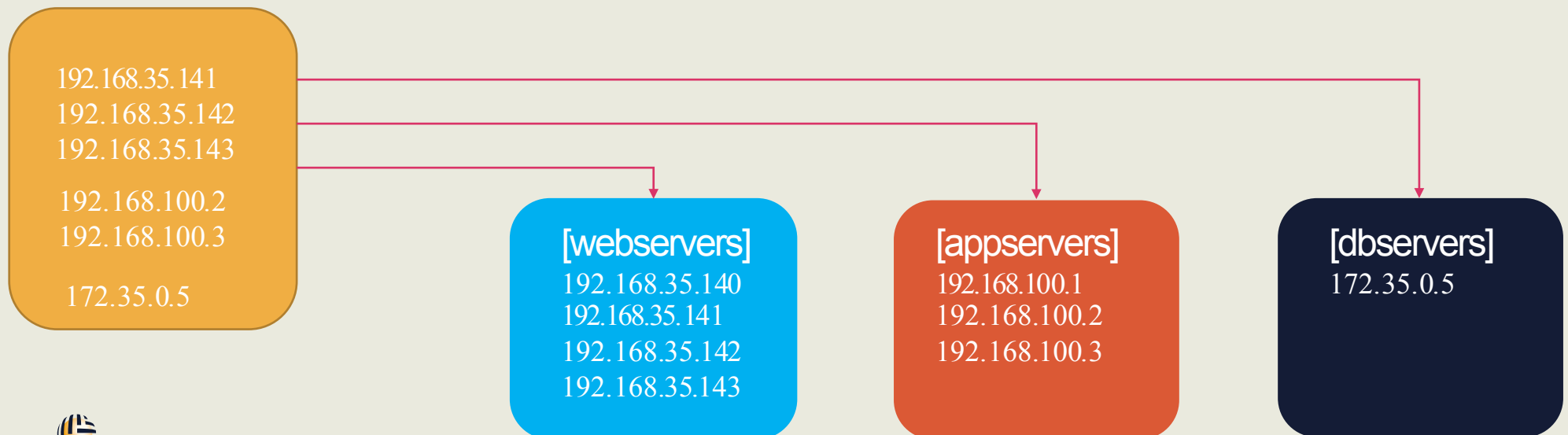
- Ansible a besoin d'identifier la liste des hôtes à gérer avant l'exécution des tâches.
- Ansible utilise un fichier d'inventaire qui lui indique les IPs ou URIs de ces hôtes
- Le fichier d'inventaire par défaut est **/etc/ansible/hosts**
- La commande suivante affiche tous les détails de l'inventaire utilisé
\$ `ansible-inventory -list <inventory_file>`



Ansible Inventory: Host

Un hôte est simplement une machine distante gérée par Ansible.

Les hôtes peuvent avoir des variables individuelles qui leur sont assignées, et peuvent également être organisés en groupes.



Ansible Inventory: Groups

Un groupe se compose de plusieurs hôtes affectés à un pool qui peuvent être facilement ciblés ensemble, ainsi que des variables qu'ils partagent en commun.

[webservers]

192.168.35.140
192.168.35.141
192.168.35.142
192.168.35.143

[appservers]

192.168.100.1
192.168.100.2
192.168.100.3

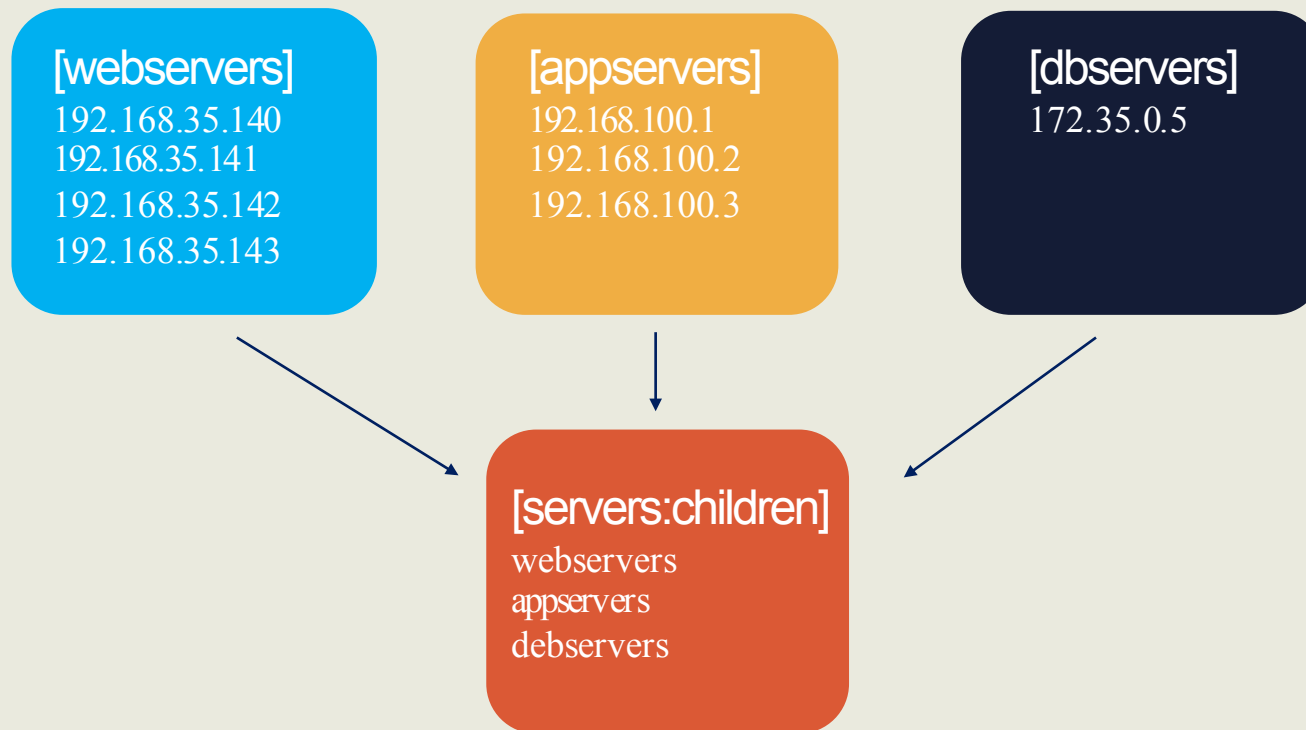
[dbservers]

172.35.0.5



Ansible Inventory: Children Groups

Un groupe peut se composer d'autres groupes.



Ansible Inventory: Host & Group

Different SSH port:

```
web1.example.com:2222
```

Attribuer des alias à des hôtes et utiliser des variables Ansible

```
web2 ansible_port=2222 ansible_host=192.168.35.102
```

Intervalle:

```
[webservers]  
web[01:50].example.com
```



Ansible Inventory: Host & Group Variables

Attribuer des variables aux hôtes qui seront utilisées plus tard dans les playbooks

```
[webservers]
web1 http_port=80 https_port=443
web2 http_port=8080 https_port=8443
```

Les variables peuvent également être appliquées à un groupe entier à la fois

```
[webservers:vars]
ntp_server=fr.pool.ntp.org
proxy=proxy.example.com
```



Ansible Inventory: Host & Group Variables

Utilisateur par défaut

`ansible_user`

- Le nom d'utilisateur à utiliser par défaut.

`ansible_pass`

- Le mot de passe à utiliser par défaut.

`ansible_connection`

- Le type de connexion
 - local
 - ssh



Ansible Inventory: Host & Group Variables

Connexion SSH :

- `ansible_ssh_host` : Nom de l'hôte.
- `ansible_ssh_port` : Numéro du port (si différent de 22).
- `ansible_ssh_user` : Nom de l'utilisateur.
- `ansible_ssh_pass` : Password à utiliser.
- `ansible_ssh_private_key_file` : fichier de clé privée à utiliser.



Ansible Inventory: Host & Group Variables

Escalade de privilèges

- `ansible_become` : Equivalent de `ansible_sudo`.
- `ansible_become_method` : Permet de définir la méthode d'escalade de privilège.
- `ansible_become_user` : Permet de définir l'utilisateur à devenir.
- `ansible_become_pass` : Permet de définir le mot de passe élevant les privilèges utilisateur.



Ansible Inventory: Host & Group Variables

Paramètres d'environnement d'hôte distant :

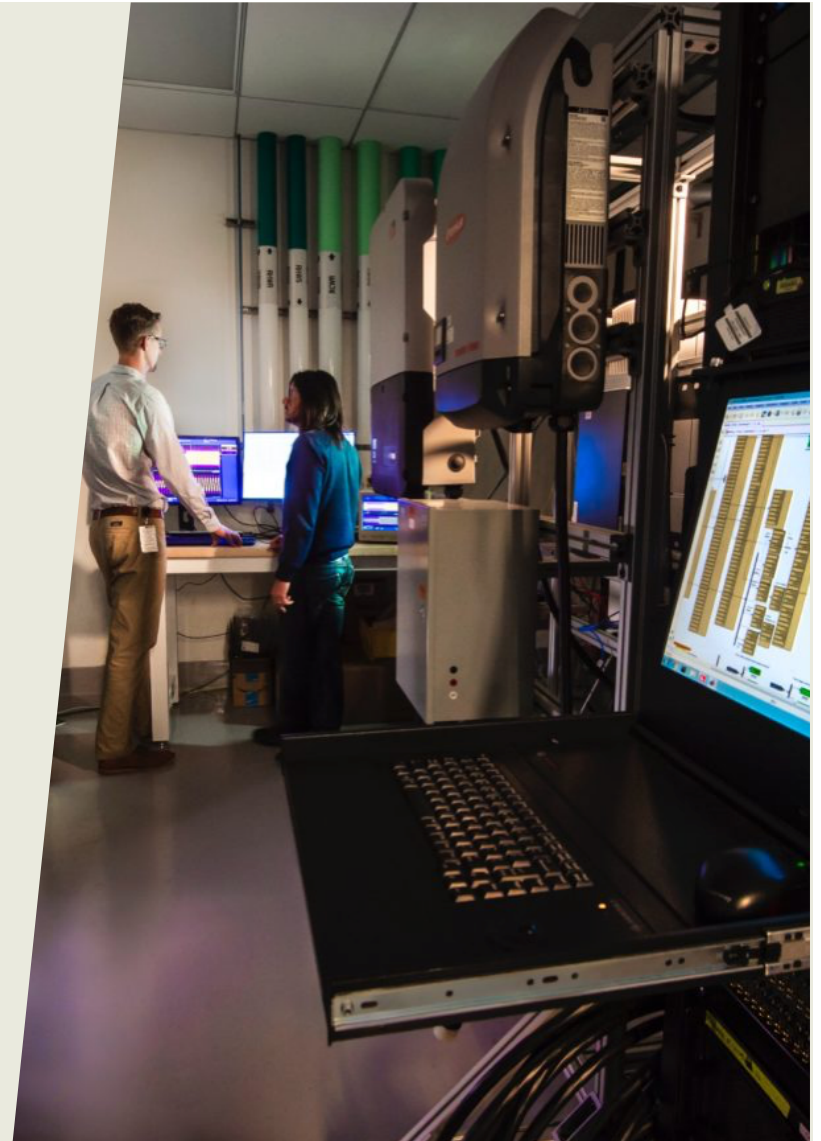
- `ansible_shell_type` : Type de shell distant.
- `ansible_python_interpreter` : L'interpréteur python de l'hôte distante.



Lab# 2: Ansible Configuration



Global Knowledge®
a skillsoft company



Ansible: Les commandes ad-hoc



Global Knowledge®
a skillsoft company

Ansible Tasks

Un Task ou une tâche est une action discrète qui est une déclaration sur l'état d'un système.

Exemples de tâches:

- ▶ Le répertoire doit exister
 - ▶ Le package doit être installé
 - ▶ Le service doit être en cours d'exécution
 - ▶ L'instance cloud doit exister
-
- Les tâches Ansible se basent sur l'exécution des modules sur les nœuds gérés



Ansible Tasks

Les modules

Les modules sont les bouts de code copiés sur le système cible pour être exécutés pour satisfaire la déclaration de tâche.

- Le code n'a pas besoin d'exister sur l'hôte distante, il est possible de le copier
- De nombreux modules sont livrés avec Ansible
- Les modules personnalisés peuvent être développés facilement
- Les modules Command / Shell existent pour des commandes simples
- Le module Script existe pour utiliser le code existant
- Le module Raw existe pour exécuter des commandes brutes sur ssh



Ansible Tasks

Les modules

Liste des modules et documentation accessible par ansible-doc:

- `ansible-doc -l`
- `ansible-doc apt`

L'indexe des modules :

- https://docs.ansible.com/ansible/2.9/modules/modules_by_category.html



Ansible Tasks

Exécution parallèle

- Par défaut, Ansible exécutera vos commandes en parallèle, en utilisant plusieurs **FORKs** de processus, de sorte que la commande se terminera plus rapidement.
- Si vous gérez quelques serveurs, cela peut ne pas être beaucoup plus rapide que d'exécuter la commande en série, sur un serveur après l'autre, mais même en gérant 5 à 10 serveurs, vous remarquerez une accélération spectaculaire si vous utilisez le parallélisme d'Ansible (qui est activé par défaut).
- Pour contrôler le nombre de forks:
 - Utilisez l'option '-f' dans la ligne de commande
 - `ansible multi -a "hostname" -f 1`
 - Utilisez la configuration « fork » dans les fichiers de configuration
 - `forks = 3`



Commandes Ad-hoc

- Une commande ad-hoc est une tâche Ansible unique à effectuer rapidement, mais on ne souhaite pas l'enregistrer pour plus tard.
- La commande ad-hoc exécute un module avec l'option **-m**
`$ ansible <cible> -m <module> -a "<args>"`
- Le module '**command**' est exécuté par défaut en cas de non précision de module.



Commandes Ad-hoc

Exemples de commandes ad-hoc

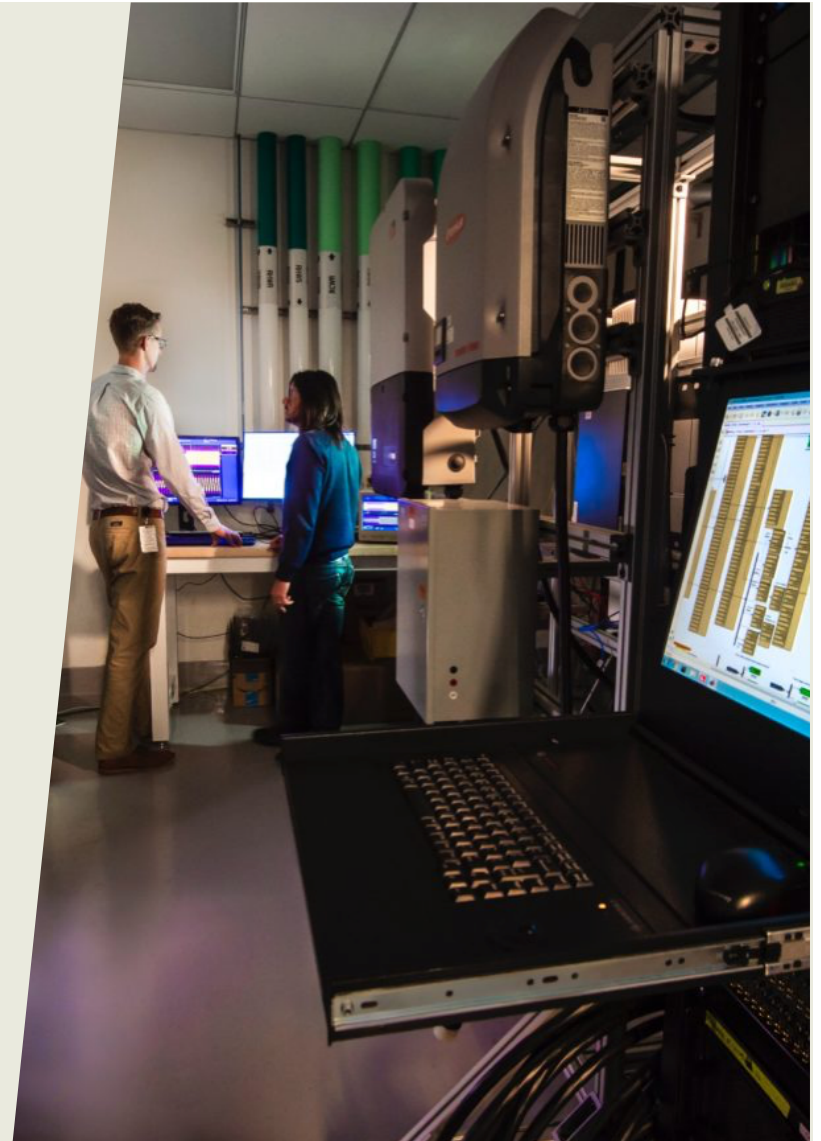
- Check all my inventory hosts are ready to be managed by Ansible
 - `$ ansible all -m ping`
- Collect and display the discovered facts for the localhost
 - `$ ansible localhost -m setup`
- Run the uptime command on all hosts in the web group
 - `$ ansible web -m command -a "uptime"`
 - `$ ansible web -a "uptime"`
- Uptime off all the machines
 - `$ ansible all -m shell -a "uptime"`



Lab# 3: Les commandes Ad-Hoc



Global Knowledge®
a skillsoft company



Ansible: Les Playbooks



Global Knowledge®
a skillsoft company

Plays & Playbooks

- Les **Plays** sont des ensembles ordonnés de tâches à exécuter par rapport à des sélections d'hôtes de votre inventaire.
- Un **Playbook** est un fichier contenant un ou plusieurs **Plays**.



Ansible playbook

- Langage de configuration, de déploiement et d'orchestration d'Ansible
- Ecrit en YAML
- Définit de manière **déclarative** vos configurations
- Lisible par l'homme et sont développés dans un langage textuel
- Les modules Ansible sont utilisés dans le Playbook pour exécuter l'opération.
- Adapté au déploiement d'applications complexes.
- Plus susceptible d'être conservé dans le système de contrôle de version



Ansible playbook

- Assure le Suivi de la configuration des serveurs
- Un playbook est un scénario décrivant les actions qui seront réalisées par les serveurs
 - les paquets à installer,
 - les fichiers à créer,
 - les services à démarrer ou arrêter,
 - ...
- **Faut faire attention à l'ordre d'exécution des tâches.**



Structure de Play

Hosts

- list

Variables

- list

Tasks

- list

Handlers

- list

- name: install and start apache

hosts: web

become: yes

vars:

http_port: 80

tasks:

- name: install nginx server

apt: pkg=nginx state=installed

notify:

- start nginx

handlers:

- name: start nginx

service: name=nginx state=started



Global Knowledge
a skillsoft company

Structure de Playbook

Dénomination significative

Nom de Play

Nom de tâche

- name: install and start apache

hosts: web

become: yes

vars:

http_port: 80

tasks:

- name: httpd package is present

yum:

name: httpd

state: latest

- name: latest index.html file is present

copy:

src: files/index.html

dest: /var/www/html/



Global Knowledge®
a skillsoft company

Structure de Playbook

Sélection des hôtes cibles

```
---  
- name: install and start apache  
  hosts: web  
  become: yes  
  vars:  
    http_port: 80  
  tasks:  
    - name: httpd package is present  
      yum:  
        name: httpd  
        state: latest  
    - name: latest index.html file is present  
      copy:  
        src: files/index.html  
        dest: /var/www/html/
```



Structure de Playbook

Escalade de privilèges (sudo)

```
---  
- name: install and start apache  
  hosts: web  
  become: yes  
  vars:  
    http_port: 80  
  tasks:  
    - name: httpd package is present  
      yum:  
        name: httpd  
        state: latest  
    - name: latest index.html file is present  
      copy:  
        src: files/index.html  
        dest: /var/www/html/
```



Structure de Playbook

Variables des plays

```
---  
- name: install and start apache  
  hosts: web  
  become: yes  
  vars:  
    http_port: 80  
  tasks:  
    - name: httpd package is present  
      yum:  
        name: httpd  
        state: latest  
    - name: latest index.html file is present  
      copy:  
        src: files/index.html  
        dest: /var/www/html/
```



Structure de Playbook

Tâches (tasks)

Exécution de module

```
---  
- name: install and start apache  
  hosts: web  
  become: yes  
  vars:  
    http_port: 80  
  tasks:  
    - name: httpd package is present  
      yum:  
        name: httpd  
        state: latest  
    - name: latest index.html file is present  
      copy:  
        src: files/index.html  
        dest: /var/www/html/
```



Global Knowledge®
a skillsoft company

Exécution de playbook

- Afficher l'aide

```
$ ansible-playbook --help
```

- Lancer le playbook script.yml, attention il faut que le playbook soit dans le même dossier

```
$ ansible-playbook script.yml
```

- Lancer le playbook script.yml peu importe où vous vous trouvez dans l'arborescence de votre poste

```
$ ansible-playbook /path/to/script.yml
```



Exécution de playbook

Vérifier la syntaxe et effectuer des essais à sec

- La vérification de la syntaxe est extrêmement simple et ne nécessite que quelques secondes pour des playbooks encore plus grands et plus complexes avec des dizaines ou des centaines d'inclusions.
 - `ansible-playbook my-playbook.yml --syntax-check`
- L'exécution d'un playbook en mode vérification est plus complexe, car Ansible exécute l'intégralité du playbook sur votre infrastructure en direct, mais sans effectuer de modifications. Au lieu de cela, Ansible met en évidence les tâches qui auraient entraîné un changement pour montrer ce qui se passera lorsque vous exécuterez le playbook plus tard.



Exécution de playbook

Dry run: essais à sec ou check mode

- Exécuté avec l'option **--check**
- Ansible exécute l'intégralité du playbook sur votre infrastructure en direct, mais sans effectuer de modifications.
- Ansible met en évidence les tâches qui auraient entraîné un changement pour montrer ce qui se passera lorsque vous exécuterez le playbook plus tard.
- Les tâches qui se basent sur les changements prévus des tâches précédentes peuvent échouer
- Lors de l'utilisation du mode `--check`, il peut être nécessaire de forcer l'exécution de certaines tâches pour s'assurer que le playbook se termine avec succès :

```
- name: A task that runs all the time, even in check mode.  
  command: mytask --option1 --option2  
  register: my_var  
  check_mode: no
```



Exécution de playbook

Les options d'exécution

- Ansible prend par défaut le fichier inventaire `/etc/ansible/hosts`
- Avec l'option (**-i**), vous sélectionnez un inventaire spécifique pour l'exécution du playbook:

```
ansible-playbook -i mesjolishosts script.yml
```

- Ajouter des variables supplémentaires au moment de l'exécution avec (**-e**) ou (**-extra-vars**)

```
ansible-playbook -i mesjolishosts script.yml -e MAVARIABLE=VALEUR
```

- Afficher la trace d'exécution

- **-v** verbose mode,
- **-vvv** encore plus de verbosité,
- **-vvvv** pour déboguer

```
ansible-playbook script.yml -vvv
```

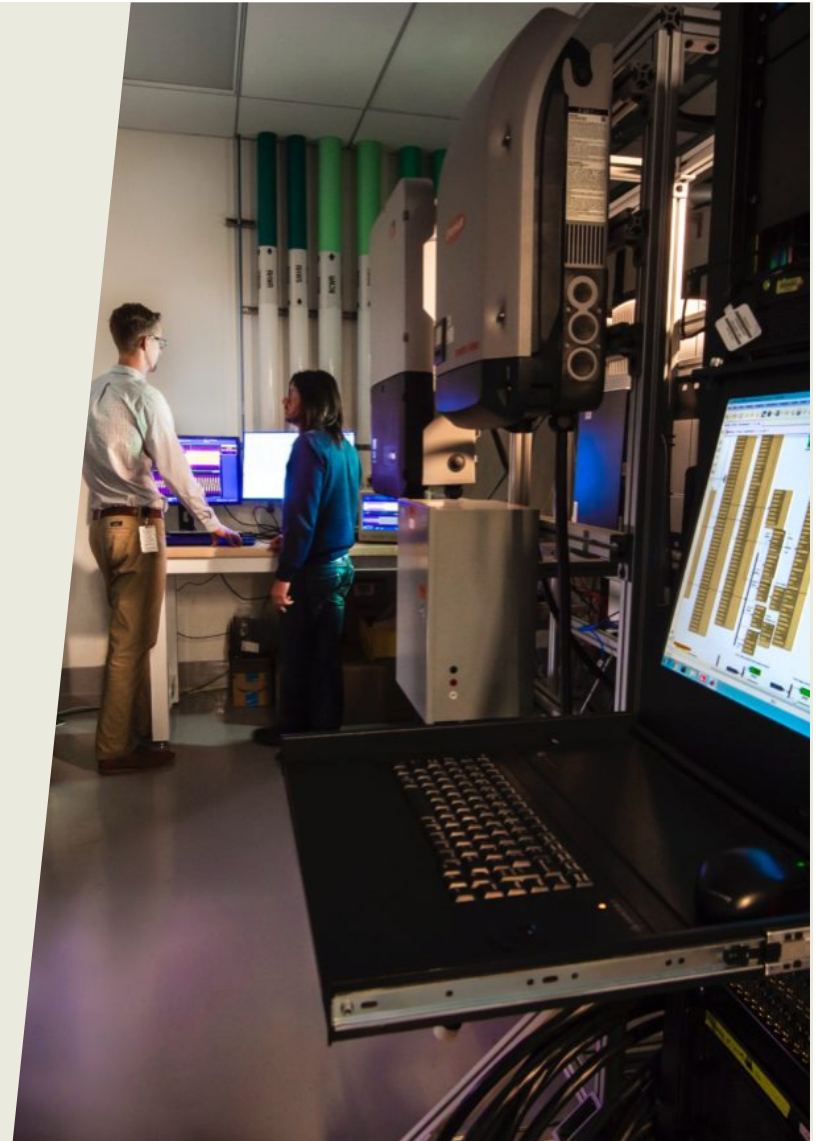


Lab# 4

Ansible Playbooks



Global Knowledge®
a skillsoft company



Les variables



Global Knowledge®
a skillsoft company

Les variables

- Les variables commencent toujours par une lettre ([A-Za-z]) et peuvent inclure n'importe quel nombre de traits de soulignement (_) ou de chiffres ([0-9]).
- Dans un fichier d'inventaire, la valeur d'une variable est affectée à l'aide d'un signe égal, comme ceci :
 - `foo=bar`
- Dans un playbook ou un fichier d'inclusion de variables, la valeur d'une variable est attribuée à l'aide de deux points, comme ceci :
 - `foo: bar`



Les variables

Il existe de nombreuses manières différentes de définir des variables à utiliser dans les tâches.

- Les variables peuvent être transmises via la ligne de commande, lors de l'appel de ansible-playbook, avec l'option `--extra-vars` :
 - `ansible-playbook example.yml --extra-vars "foo=bar"`
- Les variables peuvent être incluses en ligne avec le reste d'un playbook, dans une section **vars** :

```
---  
- hosts: example  
  vars:  
    foo: bar  
  tasks:
```

- Les variables peuvent également être incluses dans un fichier séparé, en utilisant la section **vars_files** :

```
---  
- hosts: example  
  var_files:  
    - vars.yml  
  tasks:
```

```
---  
# Variables file 'vars.yml' in the same folder as the playbook.  
foo: bar
```



Les variables

Les prompts

- Lors de l'exécution d'un playbook, vous souhaitez peut-être demander à l'utilisateur certaines entrées, et vous pouvez le faire avec la section '**vars_prompt**'.

```
---  
- hosts: all  
  remote_user: root  
  vars_prompt:  
    - name: "name"  
      prompt: "what is your name?"  
    - name: "favcolor"  
      prompt: "what is your favorite color?"
```

- Au moment de l'exécution, ansible demandera la valeur de la variable **name** en affichant le texte dans l'attribut "prompt"



Les variables

Les variables d'inventaire

- Des variables peuvent également être ajoutées via des fichiers d'inventaire Ansible,
- soit en ligne avec la définition d'hôte,

```
# Host-specific variables (defined inline).  
[paris]  
app1.example.com proxy_state=present  
app2.example.com proxy_state=absent
```

- soit après un groupe :

```
# Variables defined for the entire group.  
[paris:vars]  
cdn_host=paris.static.example.com  
api_version=3.0.1
```



Les variables

Les variables d'inventaire - group_vars & host_vars

- Au lieu de pas stocker de variables dans l'inventaire,
- Utiliser les fichiers de variables **group_vars** et **host_vars** dans un chemin spécifique, et Ansible les affectera à des hôtes et des groupes individuels définis dans votre inventaire.
- Par exemple,
 - pour appliquer un ensemble de variables à l'hôte *app1.example.com*, créez un fichier **host_vars/app1.example.com**
 - Pour appliquer un ensemble de variables à l'ensemble du groupe *paris*, créez un fichier **group_vars/paris**



Les variables

Les variables enregistrées

- Récupérer le code de retour, stderr ou stdout d'une tâche pour déterminer s'il faut exécuter une tâche ultérieure.
- Ansible vous permet d'utiliser **register** pour stocker la sortie d'une commande particulière dans une variable au moment de l'exécution.
- Les variables enregistrées peuvent être
 - des variables simples,
 - des variables de liste,
 - des variables de dictionnaire ou
 - des structures de données imbriquées complexes.

```
- name: "Node: Check list of Node.js apps running."  
  command: forever list  
  register: forever_list
```

`forever_list.stdout`

`forever_list.stderr`



Les variables

Les facts

- Par défaut, chaque fois que vous exécutez un playbook Ansible, Ansible collecte d'abord des informations ("faits") sur chaque hôte qu'on appelle Facts

```
$ ansible-playbook playbook.yml
PLAY [group] *****
GATHERING FACTS *****
ok: [host1]
```

- Les facts telles que les adresses IP d'hôte, le type de processeur, l'espace disque, les informations du système d'exploitation et les informations sur l'interface réseau
- vous pouvez utiliser les informations recueillies pour ajuster l'exécution de certaines ou pour modifier certaines informations utilisées dans les fichiers de configuration.
- Le module **setup** est responsable de la collection des facts
- Pour échapper cette étape, vous pouvez définir **gather_facts: no** dans votre playbook



Les variables

Les variables – accéder à la valeur

- Accéder à la valeur d'une variable simple (collectées par Ansible, définies dans des fichiers d'inventaire ou définies dans des fichiers playbook ou variables) avec `{{ variable }}`. Par exemple:

```
- command: /opt/my-app/rebuild {{ my_environment }}
```

- Les variables structurées sous forme de tableaux (ou « listes »), vous pouvez accéder à n'importe quelle partie du tableau en parcourant les clés du tableau, soit en utilisant les crochets (`[ ]`) ou le point (`.`)

```
foo_list:  
- one  
- two  
- three
```

```
{{ foo[0] }} => one
```

```
eth0 : {  
  "ipv4": {  
    "address": "10.0.2.15",  
    "netmask": "255.255.255.0",  
    "network": "10.0.2.0"  
  }  
},
```

```
{{ansible_facts.eth0.ipv4.address }}  
{{ ansible_facts["eth0"]["ipv4"]["address"] }}
```

```
=> 10.0.2.15
```



Les variables

Les variables – accéder à la valeur

- Exemple de **register**

```
- hosts: all
tasks:
- name: Ansible register variable basic example
  shell: "find *.txt"
  args:
    chdir: "/Users/mdtutorials2/Documents/Ansible"
  register: find_output - debug: var: find_output
```



```
output=====
ok: [localhost] => {
  "find_output": {
    "changed": true,
    "cmd": "find *.txt",
    "delta": "0:00:00.008597",
    "end": "2021-09-30 15:07:15.940235",
    "rc": 0,
    "start": "2021-09-30 15:07:15.931638",
    "stderr": "",
    "stderr_lines": [],
    "stdout": "check.txt\ncheck2.txt",
    "stdout_lines": [
      "check.txt",
      "check2.txt"
    ]
  }
}
```

```
{{ find_output.stdout }} => "check.txt\ncheck2.txt"
```

```
{{ find_output.stdout_lines }} => [ "check.txt", "check2.txt" ]
```



Les variables

Précédence des Variables

- 1. --extra-vars passed in via the command line (these always win, no matter what).
- 2. Task-level vars (in a task block).
- 3. Block-level vars (for all tasks in a block).
- 4. Role vars (e.g. [role]/vars/main.yml) and vars from include_vars module.
- 5. Vars set via set_facts modules.
- 6. Vars set via register in a task.
- 7. Individual play-level vars: 1. vars_files 2. vars_prompt 3. vars
- 8. Host facts.
- 9. Playbook host_vars.
- 10. Playbook group_vars.
- 11. Inventory: 1. host_vars 2. group_vars 3. vars
- 12. Role default vars (e.g. [role]/defaults/main.yml).

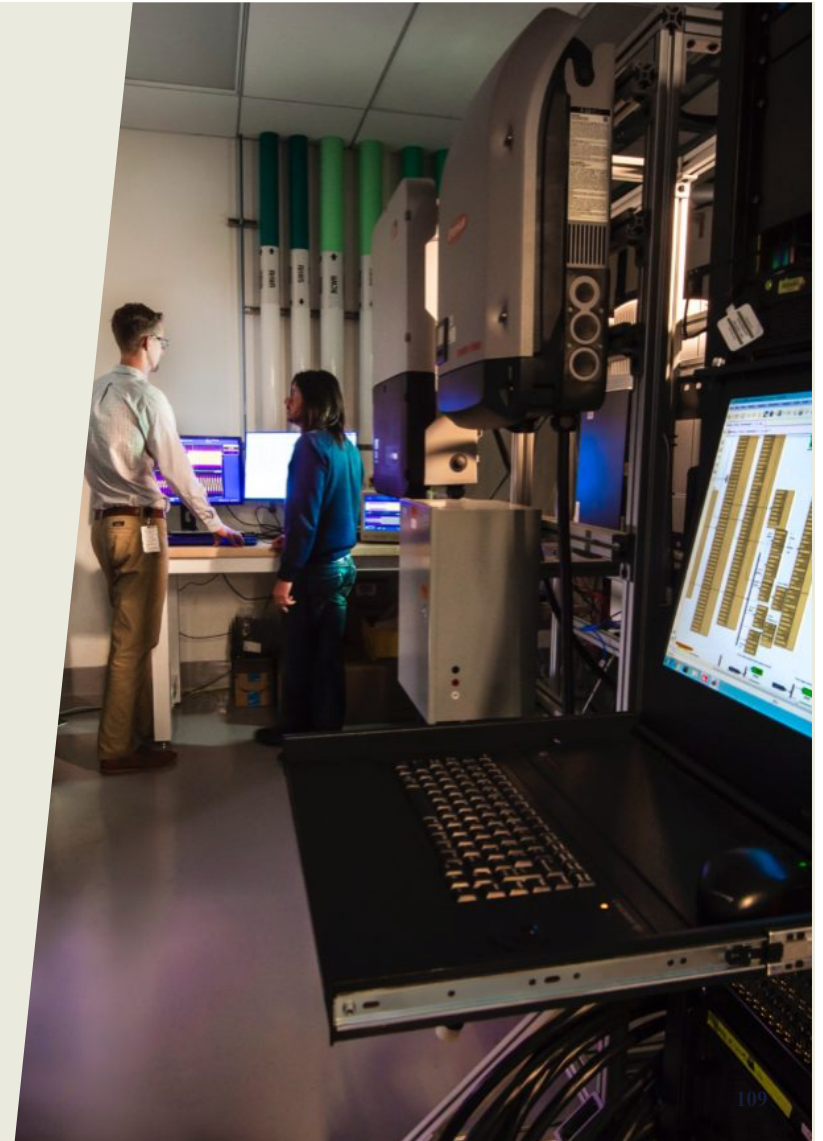


Lab #5

Les variables



Global Knowledge®
a skillsoft company



Playbooks: plus de fonctionnalités

- Loops
- Conditionals
- Templates
- Tags
- Blocks



Global Knowledge®
a skillsoft company

Loops

Les boucles permettent d'exécuter une seule tâche sur plusieurs choses, telles que

- créer de nombreux utilisateurs,
- installer de nombreux packages
- ou répéter une étape d'interrogation jusqu'à ce qu'un certain résultat soit atteint.

```
- yum:  
  name: "{{ item }}"  
  state: latest  
  loop:  
    - httpd  
    - mod_wsgi
```



Conditionals

- Ansible prend en charge l'exécution conditionnelle d'une tâche basée sur l'évaluation à l'exécution, d'une variable, d'un fait ou du résultat de la tâche précédente.
- Trois type de condition:
 - when
 - failed_when
 - changed_when
- Exemple

```
- yum:  
  name: httpd  
  state: latest  
  when: ansible_os_family == "RedHat"
```



Conditionals

Les opérateurs

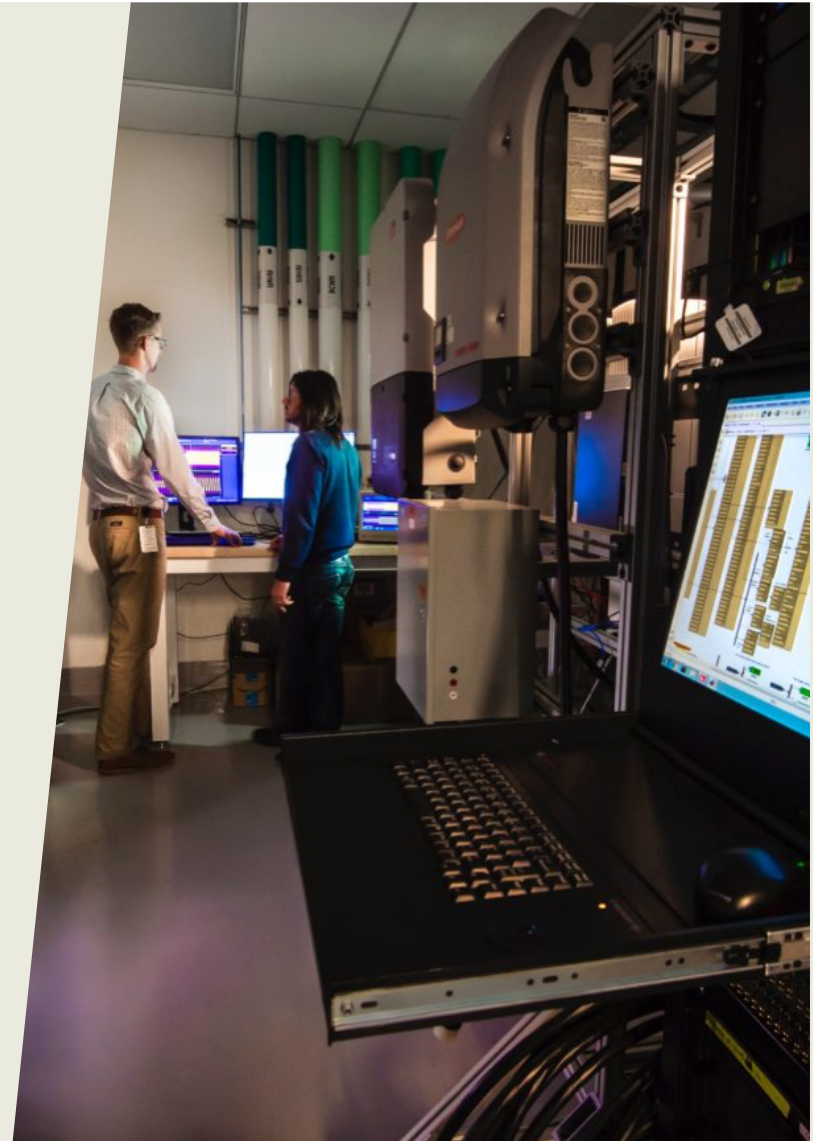
- Les opérateurs de comparaison: `==`, `!=`, `>`, `<`, `>=`, `<=`
- Les opérateurs de test: `is` (`is not`) `defined`, `undefined`, `none`, `even`, `odd`
- Les opérateurs de listes: `in`, `not in`
- Les opérateurs logiques: `and`, `or`
- Exemples:
 - `when: x == y`
 - `when: "'not installed' not in result.stdout"`
 - `when: result.stdout is defined`



Lab #6: Les contrôles



Global Knowledge®
a skillsoft company



Templates

Ansible intègre le moteur de création de modèles **Jinja2** qui peut être utilisé pour:

- Définir et modifier les variables dans les plays
- Logique conditionnelle
- Générer des fichiers tels que des configurations à partir de variables



Templates

Jinja 2

un moteur de gabarit Python

utilisé pour créer des formats de balisage HTML, XML ...

conçu pour être flexible, rapide et sécurisé.

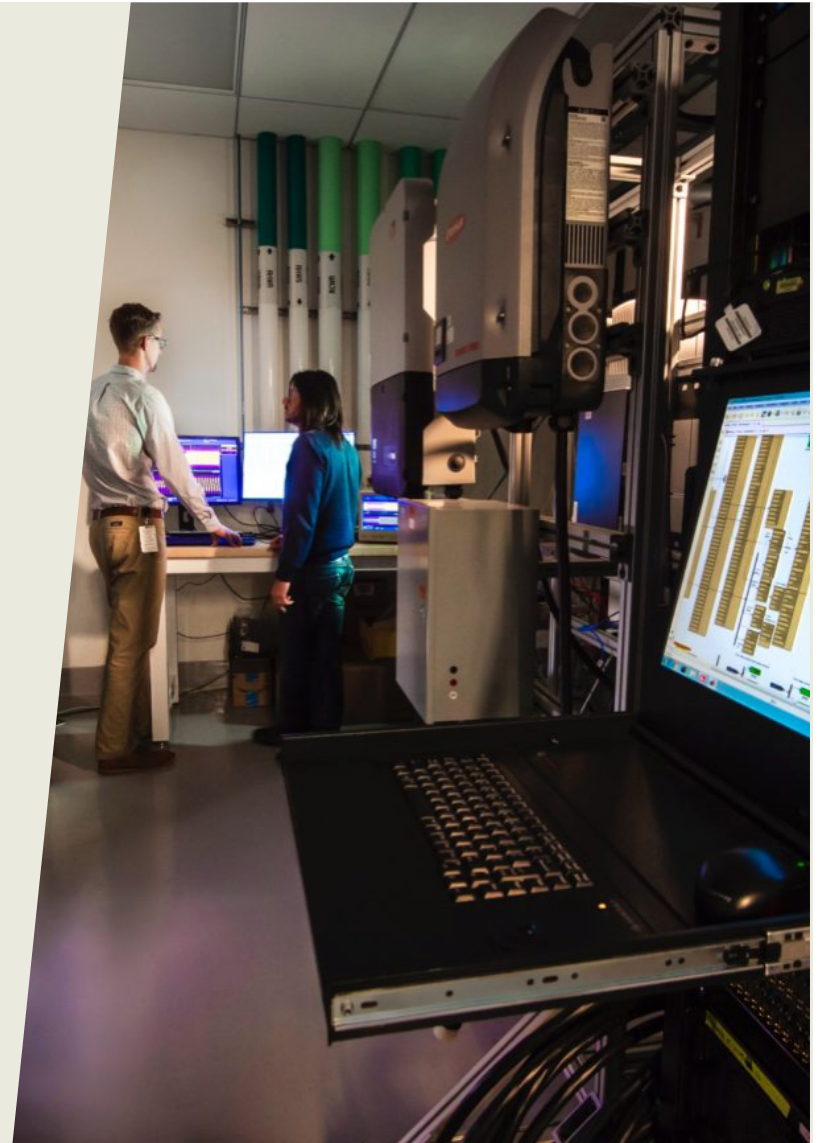
Ajouter des fonctionnalités utiles pour les environnements de modèles (boocles, tests, filtres ...)



Lab #7: Les Templates



Global Knowledge®
a skillsoft company



Les inclusions

Permet de scinder des tâches en différents fichiers et de les importer au besoin dans vos playbooks.

Sans l'include → répéter dans tous les playbooks

Avec l'include une simple référence suffit.



Les inclusions

Définir les tâches dans un fichier dédié

```
---  
- name: Install monitoring agent  
  apt:  
    name: blabla  
  
# on ajoute toutes les tâches que l'on veut
```



Les inclusions

- Ce fichier sera intégré directement dans un playbook
- Pour référencer les tasks: plaçons le directement dans la liste de tâches.
- voilà à quoi ressemblerait le playbook.

```
---  
- hosts: dbservers  
  vars:  
    email: mon@email.fr  
  
  vars_prompt:  
    - name: "dbrootpasswd"  
      prompt: "Database root password"  
  
  tasks:  
    - include: tasks/commonSetup.yml  
    - include: tasks/setupMonitoring.yml  
    - include: tasks/installMySQL.yml  
  
  # on peut bien entendu mélanger des includes et des tâches classiques  
  - name: Install htop  
    apt:  
      name: htop
```



Blocks

- Introduits dans Ansible 2.0.0
- les blocs permettent de regrouper des tâches connexes et d'appliquer des paramètres de tâche particuliers au niveau du bloc.

```
- hosts: web
  tasks:
    # Install and configure Apache on RHEL/CentOS hosts.
    - block:
        - yum: name=httpd state=present
        - template: src=httpd.conf.j2 dest=/etc/httpd/conf/httpd.conf
        - service: name=httpd state=started enabled=yes
    when: ansible_os_family == 'RedHat'
    become: yes
```

Blocks

- les blocs permettent de gérer les erreurs à l'intérieur des blocs d'une manière similaire à la gestion des exceptions de la plupart des langages de programmation.

```
tasks:
  - block:
    - name: Script to connect the app to a monitoring service.
      script: monitoring-connect.sh
  rescue:
    - name: This will only run in case of an error in the block.
      debug: msg="There was an error in the block."
  always:
    - name: This will always run, no matter what.
      debug: msg="This always executes."
```



Les tags

- Les tags vous permettent d'exécuter (ou d'exclure) des sous-ensembles de tâches d'un playbook.
- Vous pouvez marquer des rôles, des fichiers inclus, des tâches individuelles et même des pièces entières.



Les tags

Tagger une tâche

tasks:

- name: Notify on completion.

local_action:

msg: "{{inventory_hostname}} is finished!"

tags:

- notifications
- say



Les tags

Tagger une inclusion

tasks:

- include: foo.yml

tags: foo



Les tags

Tagger un play

You can apply tags to an entire play.

- hosts: webserver

tags: deploy



Les tags

Tagger un role

You can apply tags to an entire play.

- hosts: webservers

tags: deploy

roles:

Tags applied to a role will be applied to tasks in the role.

- { role: tomcat, **tags: ['tomcat', 'app']** }



Les tags

Exploiter les tags

- **--tags** : exécuter uniquement une partie particulière d'un playbook
\$ ansible-playbook tags.yml --tags "tomcat,say"
- **--skip-tags** : exclure une ou un jeu de tâches
\$ ansible-playbook tags.yml --skip-tags "notifications"



Les lookups

- Les lookups Ansible permet de récupérer des données qui sont stockées sur le nœud où est exécuté le playbook.
- Par exemple il est possible de lire le contenu d'un fichier, de générer un mot de passe aléatoire, etc.
- Le résultat retourné par un lookup est une valeur à utiliser immédiatement (affichage) ou à stocker dans une variable.
- Pour faire appel à un lookup on utilise la syntaxe suivante: `{{lookup(lookup_name, [lookup_args])}}`
- Il existe beaucoup de lookup disponible:

```
$ ansible-doc -l -t lookup
ansible.utils.get_path Retrieve the value in a v...
ansible.utils.index_of Find the indices of items...
ansible.utils.to_paths Flatten a complex object ...
ansible.utils.validate Validate data with provid... ...
```

Les lookups

Exemples

Lire des variables d'environnement

On utilise ici le lookup **env**:

```
---
- name: Load env
  gather_facts: no
  hosts: all
  vars:
    my_env: "{{ lookup('env', 'HOME') }}"
  tasks:
    - name: show the env
      debug:
        msg: "{{ my_env }}"
```

```
ok: [localhost] => {
  "msg": "/home/stagiaire"
}
```



Les lookups

Exemples

Lire le retour d'une commande

Il est possible de récupérer la sortie d'une commande avec le lookup pipe :

```
---
- name: Load stdout of a command
  gather_facts: no
  hosts: all
  vars:
    my_stdout: "{{ lookup('pipe', 'date') }}"
  tasks:
    - name: show date
      debug:
        msg: "{{ my_stdout }}"
```

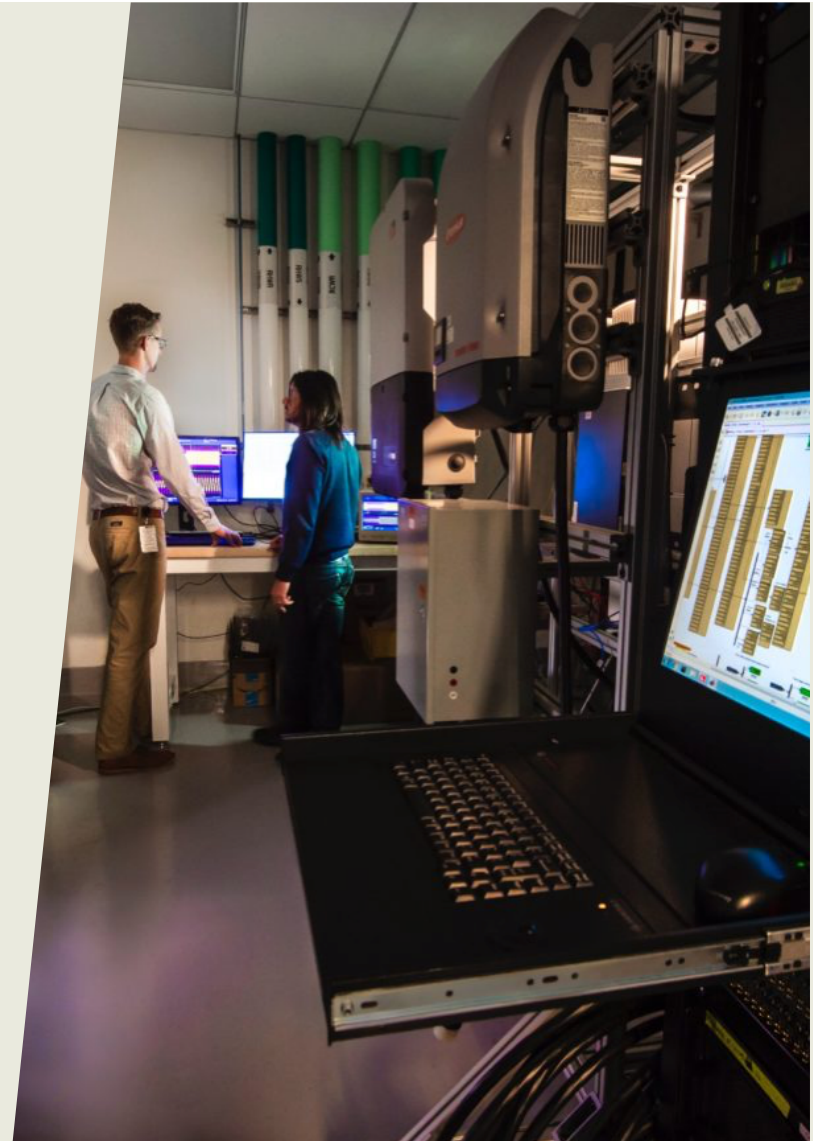
```
ok: [localhost] => {
  "msg": "jeu. 1 avr. 2021 11:01:45"
}
```



Lab #8: Manipulation des playbooks



Global Knowledge®
a skillsoft company



Ansible Roles & Galaxy



Global Knowledge®
a skillsoft company

Ansible Roles

- Les rôles sont des moyens de charger automatiquement certains fichiers vars_files, tasks et handlers en fonction d'une structure de fichiers connue.
- Le regroupement du contenu par rôles permet de partager facilement les rôles avec d'autres utilisateurs.
- Les rôles s'attendent à ce que les fichiers soient dans certains noms de répertoire.
- Les rôles doivent inclure au moins un de ces répertoires, mais il est parfaitement normal d'en exclure ceux qui ne sont pas utilisés.
- Lorsqu'il est utilisé, chaque répertoire doit contenir un fichier main.yml, qui contient le contenu pertinent



Ansible Roles - Directories

- **tasks** - contient la liste principale des tâches à exécuter par le rôle.
- **handlers** - contient des gestionnaires, qui peuvent être utilisés par ce rôle ou même n'importe où en dehors de ce rôle.
- **defaults** - variables par défaut pour le rôle
- **vars** - autres variables pour le rôle.
- **files** - contient des fichiers qui peuvent être déployés via ce rôle.
- **templates** - contient des modèles qui peuvent être déployés via ce rôle.
- **meta** - définit certaines métadonnées pour ce rôle.



Ansible Roles - Example project structure:

site.yml

roles/

common/

tasks/

handlers/

files/

templates/

vars/

defaults/

meta/

webservers/

tasks/

defaults/

meta/



Global Knowledge®
a skillsoft company

Ansible Roles – Exemple structure de projet :

D'autres fichiers YAML peuvent être inclus dans certains répertoires.

Il est courant d'inclure des tâches spécifiques à la plate-forme à partir du fichier tasks/main.yml:

```
# roles/example/tasks/main.yml
- name: added in 2.4, previously you used 'include'
  import_tasks: redhat.yml
  when: ansible_os_platform|lower == 'redhat'
- import_tasks: debian.yml
  when: ansible_os_platform|lower == 'debian'
```

```
# roles/example/tasks/redhat.yml
- yum: name="httpd" state=present
```

```
# roles/example/tasks/debian.yml
- apt: name="apache2" state=present
```



Ansible Galaxy

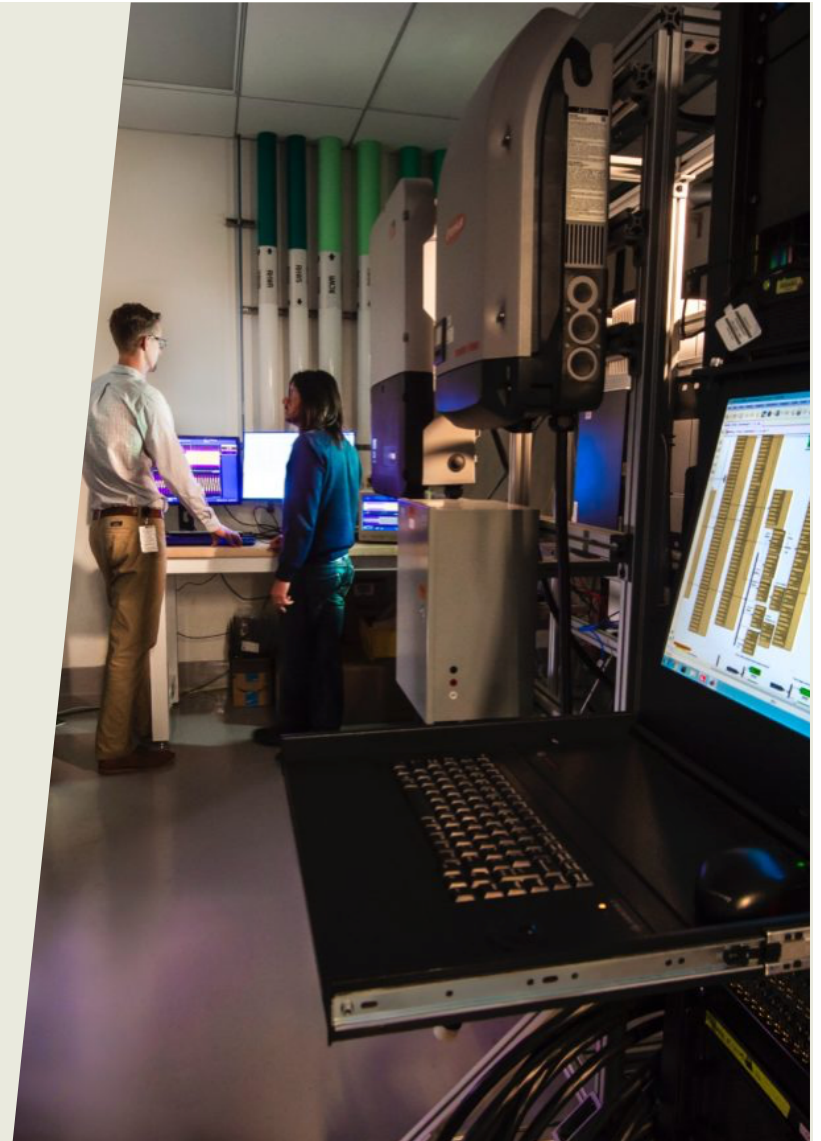
- Ansible Galaxy est un hub pour trouver, réutiliser et partager du contenu Ansible.
- Démarrez votre projet d'automatisation avec du contenu fourni et révisé par la communauté Ansible.
- <http://galaxy.ansible.com>



Lab #9: Ansible Roles



Global Knowledge®
a skillsoft company



Ansible Vault



Global Knowledge®
a skillsoft company

Ansible Vault

- « Vault » est une fonctionnalité d'ansible qui permet de conserver des données sensibles telles que des mots de passe ou des clés dans des fichiers cryptés
- Pour activer cette fonctionnalité, un outil de ligne de commande « ansible-vault » est utilisé
- Pour exécuter un playbook crypté, on utilise les options
 - --ask-vault-pass ou
 - --vault-password-file

```
ansible-playbook site.yml --ask-vault-pass
```



Ansible Vault

Commandes Vault

- Chiffrer un playbook dès sa création

```
ansible-vault create playbook_name
```

- Afficher un fichier chiffré

```
ansible-vault view playbook_name
```

- Apporter des modifications à un fichier chiffré

```
ansible-vault edit playbook_name
```



Ansible Vault

Commandes Vault

- Chiffrer un fichier non chiffré

```
ansible-vault encrypt playbook_name
```

- Déchiffrer un fichier chiffré

```
ansible-vault decrypt playbook_name
```

- Changer le mot de passe Vault

```
ansible-vault rekey playbook_name
```



Ansible Vault

Encrypter une variable

- Encrypter une variable en ligne de commande

```
ansible-vault encrypt_string password123
```

- On suppose que le mot de passe est password123.
- Ansible demande de saisir un mot de passe pour encrypter
- Le mot de passe encrypté s'affichera sous la forme suivante :

```
!vault |
$ANSIBLE_VAULT;1.1;AES256
6130656139313462323632323038313733363646431373136326236323335
3664376535633365316662613464663365303930316563380a3132323964
313164313563613761353463363833336166363962656165393936383262
3834373437333336390a6239633539393630366462326234643464663062
3165
```



Ansible Vault

Encrypter une variable

- Utiliser une variable à l'intérieur d'un playbook.
- Pouvoir afficher le mot de passe encrypté.
- Le playbook est comme suite :

```
---  
  
- name: how to use vault  
  hosts: all  
  vars:  
    my_password: !vault |  
$ANSIBLE_VAULT;1.1;AES256  
6130656139313462323632323038313733363646431373136326236323335  
3664376535633365316662613464663365303930316563380a3132323964  
313164313563613761353463363833336166363962656165393936383262  
3834373437333336390a6239633539393630366462326234643464663062  
3165  
  
  tasks:  
  
    - name: show message  
      debug:  
        msg: "My password is {{ my_password }}"
```



Ansible Vault

Encrypter une variable

- Pour exécuter le playbook qui contient la variable cryptée, ajouter l'option **–ask-vault-pass**

```
ansible-playbook -i hosts deploy.yml --ask-vault-pass
```

- Ansible demandera le mot de passe qui sera utilisé pour déchiffrer la variable

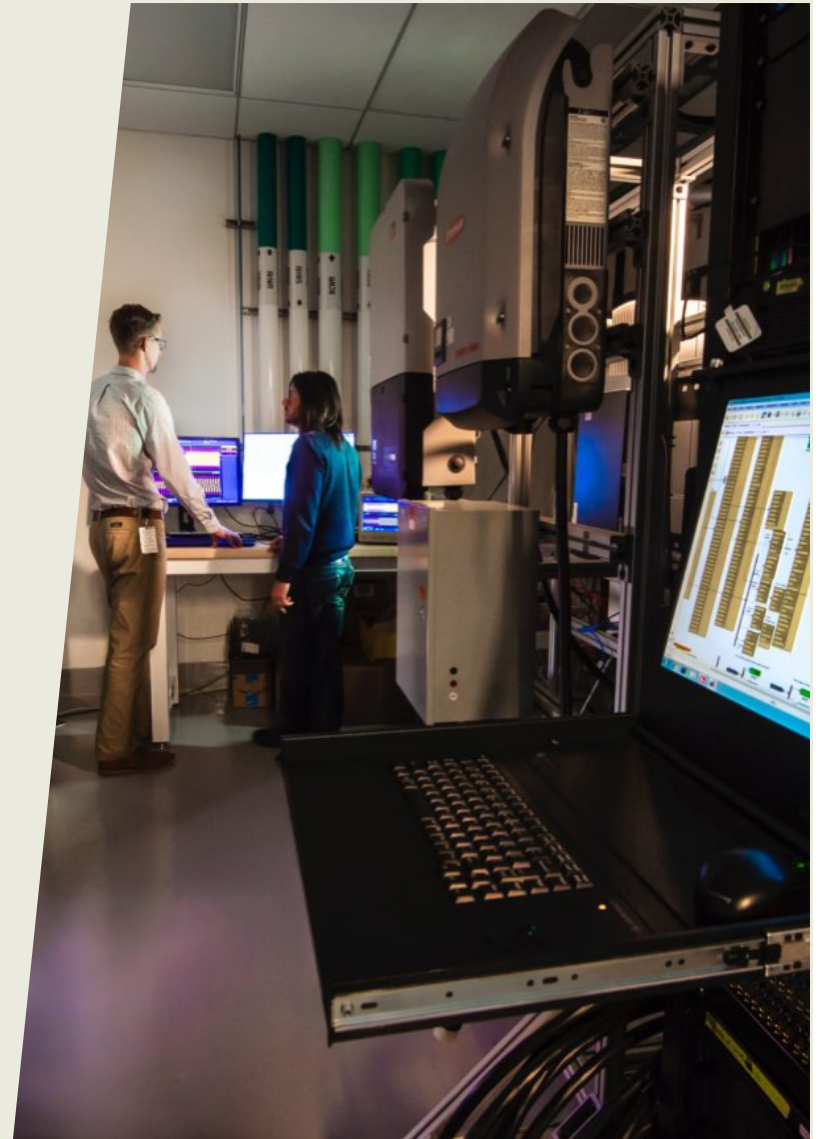


Lab #10

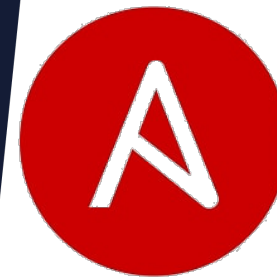
Ansible Vault



Global Knowledge®
a skillsoft company



Ansible Tower



**ANSIBLE
TOWER**
by Red Hat®



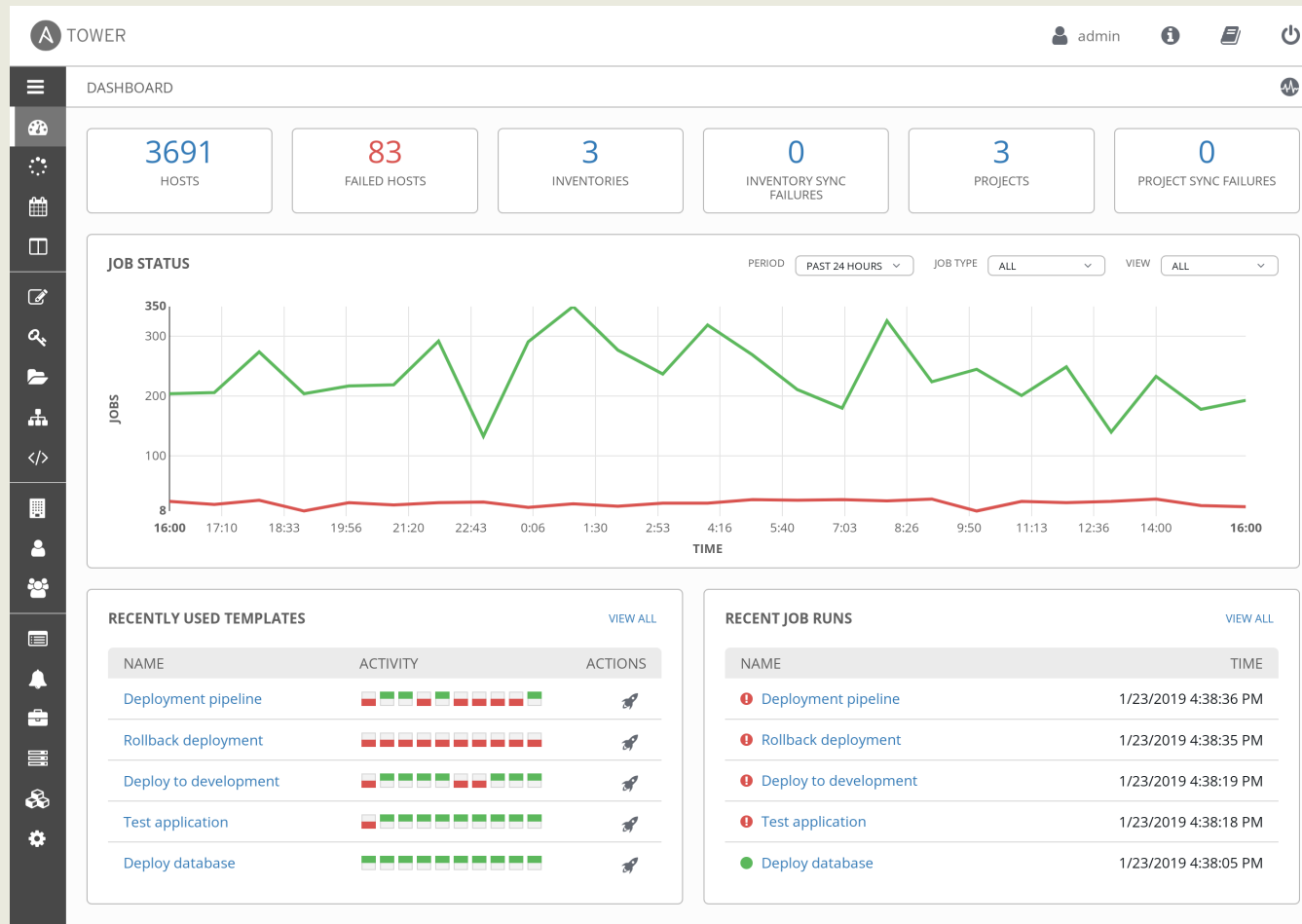
Global Knowledge®
a skillsoft company

Ansible Tower

- Ansible Tower une interface Web utilisateur facile à utiliser pour Ansible.
- Il contient les fonctions ansible les plus importantes.
- Tower centralise l'infrastructure avec contrôle d'accès basé sur les rôles
- la planification des tâches et la gestion des stocks graphique.
- L'API et le CLI REST de Tower font qu'il est facile à intégrer dans les outils existants.
- Tower est un produit payant, qui nécessite une licence.
- Une version d'essai gratuite est mise à disposition pour tester le produit.



Ansible Tower

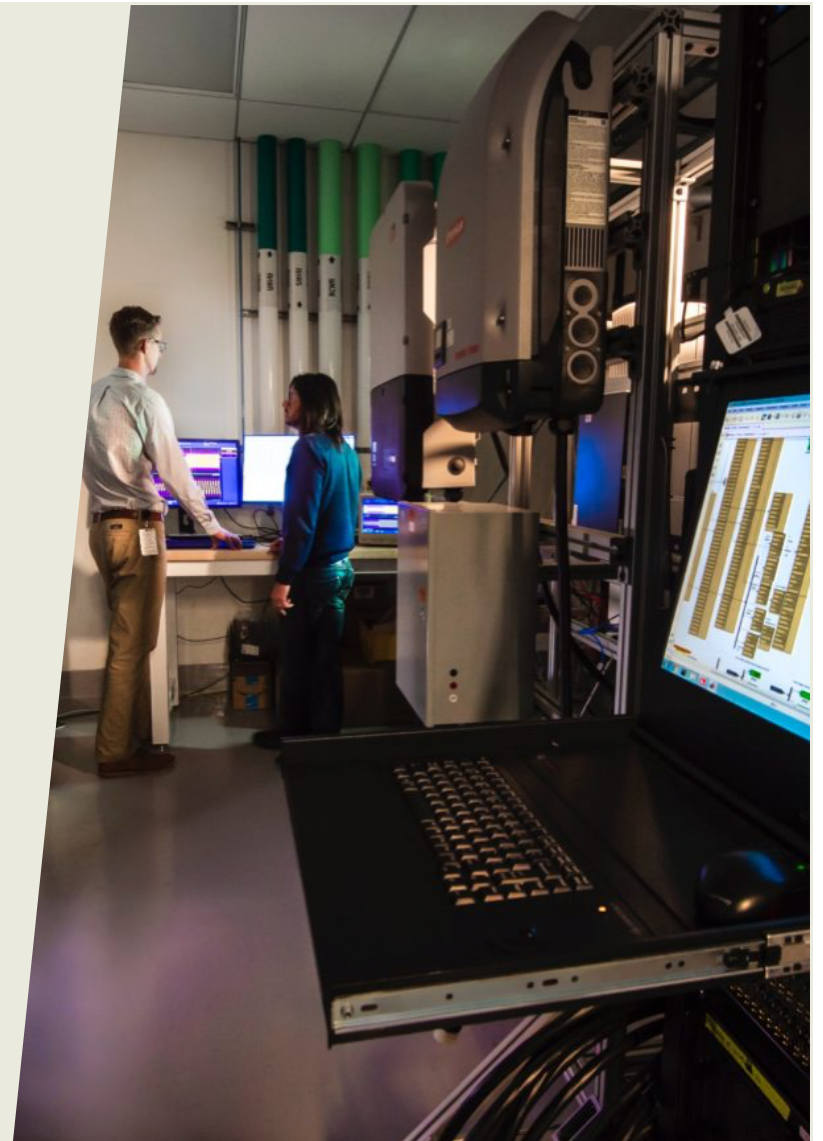


Lab #11

Ansible Tower



Global Knowledge®
a skillsoft company





Global Knowledge®
a skillsoft company

THANK YOU